Salla_Case_Study_Imad

September 21, 2025

1 Salla E-commerce Case Study — SQL + Python

Author: Imad Ghani

Deliverable: End-to-end analysis notebook with SQL (executed using SQLite), Python and interactive Plotly visuals

---

1.1 Goals

We will analyze Salla's sales data to guide product strategy. The notebook:

SQL tasks (via SQLite): 1. Top-selling products overall and by region.
2. Most popular categories.
3. Monthly, quarterly, and yearly sales (all products combined).
4. Average sale by product category and top category by customer location.

Python tasks: 1. Top 10 **stores** with highest average **daily** sales.
2. % monthly sales growth **per store**.
3. Customer **cohort analysis** using **first order date** (month). Heatmap across months by cohort.

> **Note on definitions**
> - **Sales** = sum of `order_items.price` at the item level (exclusive of freight).
> - For completeness there is **Gross (price + freight)** in some views.
> - **Store** = `seller_id` (no explicit "store" field provided).
> - **Region** = `customer_state`
> - **Order Status** only order statuses considered in progress or completed successfully are included in the analyses.
> - **Cohorts** use the **first** `order_purchase_timestamp` of a `customer_unique_id`.

1.2 0. Setup & Imports

[1]:
```
%pip install pandas plotly deep_translator
```

```
Requirement already satisfied: pandas in /Users/imadghani/GitHub/imad-
portfolio/python_venv/lib/python3.12/site-packages (2.1.4)
Requirement already satisfied: plotly in /Users/imadghani/GitHub/imad-
portfolio/python_venv/lib/python3.12/site-packages (5.24.1)
Requirement already satisfied: deep_translator in /Users/imadghani/GitHub/imad-
portfolio/python_venv/lib/python3.12/site-packages (1.11.4)
```

```
Requirement already satisfied: numpy<2,>=1.26.0 in /Users/imadghani/GitHub/imad-
portfolio/python_venv/lib/python3.12/site-packages (from pandas) (1.26.0)
Requirement already satisfied: python-dateutil>=2.8.2 in
/Users/imadghani/GitHub/imad-portfolio/python_venv/lib/python3.12/site-packages
(from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /Users/imadghani/GitHub/imad-
portfolio/python_venv/lib/python3.12/site-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.1 in /Users/imadghani/GitHub/imad-
portfolio/python_venv/lib/python3.12/site-packages (from pandas) (2025.2)
Requirement already satisfied: tenacity>=6.2.0 in /Users/imadghani/GitHub/imad-
portfolio/python_venv/lib/python3.12/site-packages (from plotly) (8.5.0)
Requirement already satisfied: packaging in /Users/imadghani/GitHub/imad-
portfolio/python_venv/lib/python3.12/site-packages (from plotly) (23.1)
Requirement already satisfied: beautifulsoup4<5.0.0,>=4.9.1 in
/Users/imadghani/GitHub/imad-portfolio/python_venv/lib/python3.12/site-packages
(from deep_translator) (4.13.3)
Requirement already satisfied: requests<3.0.0,>=2.23.0 in
/Users/imadghani/GitHub/imad-portfolio/python_venv/lib/python3.12/site-packages
(from deep_translator) (2.32.3)
Requirement already satisfied: soupsieve>1.2 in /Users/imadghani/GitHub/imad-
portfolio/python_venv/lib/python3.12/site-packages (from
beautifulsoup4<5.0.0,>=4.9.1->deep_translator) (2.6)
Requirement already satisfied: typing-extensions>=4.0.0 in
/Users/imadghani/GitHub/imad-portfolio/python_venv/lib/python3.12/site-packages
(from beautifulsoup4<5.0.0,>=4.9.1->deep_translator) (4.13.0)
Requirement already satisfied: charset-normalizer<4,>=2 in
/Users/imadghani/GitHub/imad-portfolio/python_venv/lib/python3.12/site-packages
(from requests<3.0.0,>=2.23.0->deep_translator) (3.4.1)
Requirement already satisfied: idna<4,>=2.5 in /Users/imadghani/GitHub/imad-
portfolio/python_venv/lib/python3.12/site-packages (from
requests<3.0.0,>=2.23.0->deep_translator) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/Users/imadghani/GitHub/imad-portfolio/python_venv/lib/python3.12/site-packages
(from requests<3.0.0,>=2.23.0->deep_translator) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in
/Users/imadghani/GitHub/imad-portfolio/python_venv/lib/python3.12/site-packages
(from requests<3.0.0,>=2.23.0->deep_translator) (2025.1.31)
Requirement already satisfied: six>=1.5 in /Users/imadghani/GitHub/imad-
portfolio/python_venv/lib/python3.12/site-packages (from python-
dateutil>=2.8.2->pandas) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
```

[2]:
```
import os
import sqlite3
from datetime import datetime
import pandas as pd
import numpy as np
```

```
# Plotly for interactive charts
import plotly.express as px
import plotly.graph_objects as go

import time
from deep_translator import GoogleTranslator


pd.set_option("display.max_columns", 100)
pd.set_option("display.width", 120)
print("Pandas:", pd.__version__)
```

```
Pandas: 2.1.4
```

1.3 1. Load data & basic preparation and profiling

We load the four CSVs and parse timestamps. We keep raw copies (`*_raw`) and create cleaned DataFrames.

[3]:
```
DATA_DIR = './'

orders_path = os.path.join(DATA_DIR, 'orders.csv')
order_items_path = os.path.join(DATA_DIR, 'order_items.csv')
products_path = os.path.join(DATA_DIR, 'products.csv')
customers_path = os.path.join(DATA_DIR, 'customers.csv')

orders_raw = pd.read_csv(orders_path)
order_items_raw = pd.read_csv(order_items_path)
products_raw = pd.read_csv(products_path)
customers_raw = pd.read_csv(customers_path)

def parse_dt(series):
    # Robust datetime parser for typical formats
    return pd.to_datetime(series, errors='coerce')

orders = orders_raw.copy()
orders['order_purchase_timestamp'] =␣
 ↪parse_dt(orders['order_purchase_timestamp'])
orders['order_approved_at'] = parse_dt(orders['order_approved_at'])
orders['order_delivered_carrier_date'] =␣
 ↪parse_dt(orders['order_delivered_carrier_date'])
orders['order_delivered_customer_date'] =␣
 ↪parse_dt(orders['order_delivered_customer_date'])
orders['order_estimated_delivery_date'] =␣
 ↪parse_dt(orders['order_estimated_delivery_date'])

order_items = order_items_raw.copy()
```

```
# Ensure numeric types
for col in ['price', 'freight_value']:
    if col in order_items.columns:
        order_items[col] = pd.to_numeric(order_items[col], errors='coerce')

# Parse shipping_limit_date
order_items['shipping_limit_date'] =␣
 ↪parse_dt(order_items['shipping_limit_date'])

products = products_raw.copy()
# Handle missing product categories
products['product_category_name'] = products['product_category_name'].
 ↪fillna('unknown_category')

# Handle missing product dimensions (category-specific median imputation)
for col in ['product_name_lenght', 'product_description_lenght',␣
 ↪'product_photos_qty']:
    # Calculate category-specific medians
    category_medians = products.groupby('product_category_name')[col].median()
    # Fill missing values with category-specific medians
    products[col] = products[col].fillna(products['product_category_name'].
 ↪map(category_medians))
    # If still missing (e.g., unknown_category has no data), use overall median
    products[col] = products[col].fillna(products[col].median())

# Handle zero weight products and missing weights (category-specific median␣
 ↪imputation)
# First, replace zeros with NaN to treat them as missing
products.loc[products['product_weight_g'] == 0, 'product_weight_g'] = np.nan
# Calculate category-specific medians for weight
category_weight_medians = products.
 ↪groupby('product_category_name')['product_weight_g'].median()
# Fill missing weights with category-specific medians
products['product_weight_g'] = products['product_weight_g'].
 ↪fillna(products['product_category_name'].map(category_weight_medians))
# If still missing, use overall median
products['product_weight_g'] = products['product_weight_g'].
 ↪fillna(products['product_weight_g'].median())

# Handle missing product dimensions (category-specific median imputation)
for col in ['product_length_cm', 'product_height_cm', 'product_width_cm']:
    # Calculate category-specific medians
    category_medians = products.groupby('product_category_name')[col].median()
    # Fill missing values with category-specific medians
    products[col] = products[col].fillna(products['product_category_name'].
 ↪map(category_medians))
```

```
    # If still missing, use overall median
    products[col] = products[col].fillna(products[col].median())

customers = customers_raw.copy()
# Standardize text fields
customers['customer_city'] = customers['customer_city'].str.title().str.strip()
customers['customer_state'] = customers['customer_state'].str.upper().str.
 ↪strip()

# Handle missing delivery dates for delivered orders (data consistency)
orders.loc[(orders['order_status'] == 'delivered') &
           (orders['order_delivered_customer_date'].isna()),
           'order_delivered_customer_date'] =␣
 ↪orders['order_estimated_delivery_date']

# Standardize order status
orders['order_status'] = orders['order_status'].str.lower().str.strip()

# Add data quality flags
orders['has_missing_delivery_date'] = orders['order_delivered_customer_date'].
 ↪isna()
products['has_missing_category'] = products['product_category_name'] ==␣
 ↪'unknown_category'
products['has_zero_weight'] = products['product_weight_g'] == 0

# Create freight shipment classification
# Freight criteria based on standard shipping limits:
# - Weight: >32kg (32,000g)
# - Dimensions: >274cm length OR >419cm length + girth combined
# - Girth = 2 * (width + height)

print("=== FREIGHT SHIPMENT CLASSIFICATION ===")
print()

# Calculate girth (2 * (width + height))
products['girth_cm'] = 2 * (products['product_width_cm'] +␣
 ↪products['product_height_cm'])

# Calculate length + girth combined
products['length_plus_girth_cm'] = products['product_length_cm'] +␣
 ↪products['girth_cm']

# Define freight criteria
WEIGHT_LIMIT_G = 32000  # 32kg in grams
LENGTH_LIMIT_CM = 274   # 274cm length limit
LENGTH_PLUS_GIRTH_LIMIT_CM = 419  # 419cm length + girth limit
```

```
# Create freight classification
products['is_freight_shipment'] = (
    (products['product_weight_g'] > WEIGHT_LIMIT_G) |
    (products['product_length_cm'] > LENGTH_LIMIT_CM) |
    (products['length_plus_girth_cm'] > LENGTH_PLUS_GIRTH_LIMIT_CM)
)

# Print freight classification summary
freight_count = products['is_freight_shipment'].sum()
total_products = len(products)
print(f"Products classified as freight shipments: {freight_count}
 ↪({freight_count/total_products*100:.2f}% of total products)")
print()

# Breakdown by criteria
weight_freight = (products['product_weight_g'] > WEIGHT_LIMIT_G).sum()
length_freight = (products['product_length_cm'] > LENGTH_LIMIT_CM).sum()
girth_freight = (products['length_plus_girth_cm'] > LENGTH_PLUS_GIRTH_LIMIT_CM).
 ↪sum()

print("Freight classification breakdown:")
print(f"  Weight > 32kg: {weight_freight} products")
print(f"  Length > 274cm: {length_freight} products")
print(f"  Length + Girth > 419cm: {girth_freight} products")
print()

# Show sample of freight products
freight_products = products[products['is_freight_shipment']][
    ['product_id', 'product_category_name', 'product_weight_g',
 ↪'product_length_cm',
     'product_width_cm', 'product_height_cm', 'girth_cm',
 ↪'length_plus_girth_cm']
].head(5)

if len(freight_products) > 0:
    print("Sample freight products:")
    for _, row in freight_products.iterrows():
        print(f"  {row['product_id'][:8]}... - Weight: {row['product_weight_g']:
 ↪.0f}g, "
              f"Length: {row['product_length_cm']:.0f}cm, L+G:
 ↪{row['length_plus_girth_cm']:.0f}cm")
print()

# Create comprehensive delivery analysis flag
orders_clean = orders.copy()
```

```
orders_clean['order_purchase_timestamp'] = pd.
 ↪to_datetime(orders_clean['order_purchase_timestamp'], errors='coerce')
orders_clean['order_approved_at'] = pd.
 ↪to_datetime(orders_clean['order_approved_at'], errors='coerce')
orders_clean['order_delivered_carrier_date'] = pd.
 ↪to_datetime(orders_clean['order_delivered_carrier_date'], errors='coerce')
orders_clean['order_delivered_customer_date'] = pd.
 ↪to_datetime(orders_clean['order_delivered_customer_date'], errors='coerce')
orders_clean['order_estimated_delivery_date'] = pd.
 ↪to_datetime(orders_clean['order_estimated_delivery_date'], errors='coerce')

# Flag for invalid delivery date logic
orders['is_invalid_for_delivery_analysis'] = (
    # Delivered orders missing delivery date
    ((orders_clean['order_status'] == 'delivered') &␣
 ↪orders_clean['order_delivered_customer_date'].isna()) |
    # Orders delivered before carrier pickup
    (orders_clean['order_delivered_customer_date'] <␣
 ↪orders_clean['order_delivered_carrier_date']) |
    # Orders delivered before approval
    (orders_clean['order_delivered_customer_date'] <␣
 ↪orders_clean['order_approved_at']) |
    # Orders approved before purchase
    (orders_clean['order_approved_at'] <␣
 ↪orders_clean['order_purchase_timestamp']) |
    # Orders with extreme delivery delays (>365 days)
    ((orders_clean['order_delivered_customer_date'] -␣
 ↪orders_clean['order_estimated_delivery_date']).dt.days > 365)
)

# Print count of invalid delivery analysis orders
invalid_delivery_count = orders['is_invalid_for_delivery_analysis'].sum()
total_orders = len(orders)
print(f"Orders with invalid delivery date logic: {invalid_delivery_count}␣
 ↪({invalid_delivery_count/total_orders*100:.2f}% of total orders)")
print()

# Check for missing IDs and referential integrity
print("=== MISSING ID & REFERENTIAL INTEGRITY CHECKS ===")
print()

# Check for missing primary keys
print("MISSING PRIMARY KEYS:")
print(f"Orders missing order_id: {orders['order_id'].isna().sum()}")
print(f"Order_items missing order_id: {order_items['order_id'].isna().sum()}")
```

```
print(f"Order_items missing product_id: {order_items['product_id'].isna().
  ↪sum()}")
print(f"Products missing product_id: {products['product_id'].isna().sum()}")
print(f"Customers missing customer_id: {customers['customer_id'].isna().sum()}")
print()

# Check for orphaned records (referential integrity)
print("ORPHANED RECORDS:")
orphaned_order_items = ~order_items['order_id'].isin(orders['order_id'])
orphaned_products = ~order_items['product_id'].isin(products['product_id'])
orphaned_customers = ~orders['customer_id'].isin(customers['customer_id'])

print(f"Order_items with non-existent order_id: {orphaned_order_items.sum()}")
print(f"Order_items with non-existent product_id: {orphaned_products.sum()}")
print(f"Orders with non-existent customer_id: {orphaned_customers.sum()}")
print()

# Check for duplicate primary keys
print("DUPLICATE PRIMARY KEYS:")
print(f"Duplicate order_ids: {orders['order_id'].duplicated().sum()}")
print(f"Duplicate product_ids: {products['product_id'].duplicated().sum()}")
print(f"Duplicate customer_ids: {customers['customer_id'].duplicated().sum()}")
print()

# Check for business logic violations
print("BUSINESS LOGIC CHECKS:")
print()

# Date consistency checks
orders_clean = orders.copy()
orders_clean['order_purchase_timestamp'] = pd.
  ↪to_datetime(orders_clean['order_purchase_timestamp'], errors='coerce')
orders_clean['order_approved_at'] = pd.
  ↪to_datetime(orders_clean['order_approved_at'], errors='coerce')
orders_clean['order_delivered_carrier_date'] = pd.
  ↪to_datetime(orders_clean['order_delivered_carrier_date'], errors='coerce')
orders_clean['order_delivered_customer_date'] = pd.
  ↪to_datetime(orders_clean['order_delivered_customer_date'], errors='coerce')
orders_clean['order_estimated_delivery_date'] = pd.
  ↪to_datetime(orders_clean['order_estimated_delivery_date'], errors='coerce')

# Orders approved before purchase
approved_before_purchase = (orders_clean['order_approved_at'] <␣
  ↪orders_clean['order_purchase_timestamp']).sum()
print(f"Orders approved before purchase: {approved_before_purchase}")
```

```
# Orders delivered before carrier pickup
delivered_before_carrier = (orders_clean['order_delivered_customer_date'] <␣
 ↪orders_clean['order_delivered_carrier_date']).sum()
print(f"Orders delivered before carrier pickup: {delivered_before_carrier}")

# Orders delivered before approval
delivered_before_approval = (orders_clean['order_delivered_customer_date'] <␣
 ↪orders_clean['order_approved_at']).sum()
print(f"Orders delivered before approval: {delivered_before_approval}")

# Orders with delivery date way past estimated (potential data issues)
orders_clean['delivery_delay_days'] =␣
 ↪(orders_clean['order_delivered_customer_date'] -␣
 ↪orders_clean['order_estimated_delivery_date']).dt.days
extreme_delays = (orders_clean['delivery_delay_days'] > 365).sum()
print(f"Orders with >365 day delivery delay: {extreme_delays}")

print()

# Price and freight value checks
print("PRICE & FREIGHT CHECKS:")
print(f"Negative prices: {(order_items['price'] < 0).sum()}")
print(f"Zero prices: {(order_items['price'] == 0).sum()}")
print(f"Negative freight values: {(order_items['freight_value'] < 0).sum()}")
print(f"Zero freight values: {(order_items['freight_value'] == 0).sum()}")

# Check for extreme price outliers
price_p99 = order_items['price'].quantile(0.99)
price_p01 = order_items['price'].quantile(0.01)
print(f"Prices > 99th percentile (${price_p99:.2f}): {(order_items['price'] >␣
 ↪price_p99).sum()}")
print(f"Prices < 1st percentile (${price_p01:.2f}): {(order_items['price'] <␣
 ↪price_p01).sum()}")

print()

# Product dimension checks
print("PRODUCT DIMENSION CHECKS:")
print(f"Products with zero weight: {(products['product_weight_g'] == 0).sum()}")
print(f"Products with zero dimensions: {((products['product_length_cm'] == 0) |␣
 ↪(products['product_height_cm'] == 0) | (products['product_width_cm'] == 0)).
 ↪sum()}")

# Check for unrealistic dimensions (likely data entry errors)
unrealistic_weight = (products['product_weight_g'] > 100000).sum()  # >100kg
unrealistic_length = (products['product_length_cm'] > 1000).sum()   # >10m
```

```
print(f"Products with unrealistic weight (>100kg): {unrealistic_weight}")
print(f"Products with unrealistic length (>10m): {unrealistic_length}")

print()

# Order status consistency
print("ORDER STATUS CONSISTENCY:")
print("Order status distribution:")
status_counts = orders['order_status'].value_counts()
for status, count in status_counts.items():
    print(f"  {status}: {count}")

# Check for delivered orders missing delivery dates
delivered_missing_delivery = ((orders['order_status'] == 'delivered') &
                              (orders['order_delivered_customer_date'].isna())).
 ↪sum()
print(f"Delivered orders missing delivery date: {delivered_missing_delivery}")

# Check for shipped orders missing carrier dates
shipped_missing_carrier = ((orders['order_status'] == 'shipped') &
                           (orders['order_delivered_carrier_date'].isna())).sum()
print(f"Shipped orders missing carrier date: {shipped_missing_carrier}")

print()

# Customer data checks
print("CUSTOMER DATA CHECKS:")
print(f"Customers with missing city: {customers['customer_city'].isna().sum()}")
print(f"Customers with missing state: {customers['customer_state'].isna().
 ↪sum()}")
print(f"Unique customers: {customers['customer_unique_id'].nunique()}")
print(f"Total customer records: {len(customers)}")

# Check for customers with multiple customer_ids (potential data quality issue)
customer_id_counts = customers.groupby('customer_unique_id')['customer_id'].
 ↪nunique()
multiple_customer_ids = (customer_id_counts > 1).sum()
print(f"Unique customers with multiple customer_ids: {multiple_customer_ids}")

print()

# Seller analysis
print("SELLER DATA CHECKS:")
unique_sellers = order_items['seller_id'].nunique()
total_seller_records = len(order_items)
print(f"Unique sellers: {unique_sellers}")
print(f"Total seller records: {total_seller_records}")
```

```
# Check for sellers with extreme price ranges (potential data quality issue)
seller_price_stats = order_items.groupby('seller_id')['price'].agg(['min',␣
 ↪'max', 'std'])
sellers_with_extreme_range = (seller_price_stats['max'] /␣
 ↪seller_price_stats['min'] > 1000).sum()
print(f"Sellers with extreme price ranges (>1000x min):␣
 ↪{sellers_with_extreme_range}")

print()

# Quick sanity
print("Orders:", orders.shape, "Order_items:", order_items.shape, "Products:",␣
 ↪products.shape, "Customers:", customers.shape)
```

```
=== FREIGHT SHIPMENT CLASSIFICATION ===

Products classified as freight shipments: 2 (0.01% of total products)

Freight classification breakdown:
  Weight > 32kg: 1 products
  Length > 274cm: 0 products
  Length + Girth > 419cm: 1 products

Sample freight products:
  26644690… - Weight: 40425g, Length: 13cm, L+G: 199cm
  b1780830… - Weight: 1050g, Length: 23cm, L+G: 445cm

Orders with invalid delivery date logic: 84 (0.08% of total orders)

=== MISSING ID & REFERENTIAL INTEGRITY CHECKS ===

MISSING PRIMARY KEYS:
Orders missing order_id: 0
Order_items missing order_id: 0
Order_items missing product_id: 0
Products missing product_id: 0
Customers missing customer_id: 0

ORPHANED RECORDS:
Order_items with non-existent order_id: 0
Order_items with non-existent product_id: 0
Orders with non-existent customer_id: 0

DUPLICATE PRIMARY KEYS:
Duplicate order_ids: 0
Duplicate product_ids: 0
```

```
Duplicate customer_ids: 0

BUSINESS LOGIC CHECKS:

Orders approved before purchase: 0
Orders delivered before carrier pickup: 23
Orders delivered before approval: 61
Orders with >365 day delivery delay: 0

PRICE & FREIGHT CHECKS:
Negative prices: 0
Zero prices: 0
Negative freight values: 0
Zero freight values: 383
Prices > 99th percentile ($890.00): 1117
Prices < 1st percentile ($9.99): 1080

PRODUCT DIMENSION CHECKS:
Products with zero weight: 0
Products with zero dimensions: 0
Products with unrealistic weight (>100kg): 0
Products with unrealistic length (>10m): 0

ORDER STATUS CONSISTENCY:
Order status distribution:
  delivered: 96478
  shipped: 1107
  canceled: 625
  unavailable: 609
  invoiced: 314
  processing: 301
  created: 5
  approved: 2
Delivered orders missing delivery date: 0
Shipped orders missing carrier date: 0

CUSTOMER DATA CHECKS:
Customers with missing city: 0
Customers with missing state: 0
Unique customers: 96096
Total customer records: 99441
Unique customers with multiple customer_ids: 2997

SELLER DATA CHECKS:
Unique sellers: 3095
Total seller records: 112650
Sellers with extreme price ranges (>1000x min): 0
```

```
Orders: (99441, 10) Order_items: (112650, 7) Products: (32951, 14) Customers:
(99441, 4)
```

1.4 2. Translate Portuguese Product Categories to English

We will using Google Translate through an open source Python library called deep-translator

[4]:
```
# Fully automated translation using deep-translator library
print("=== TRANSLATING PRODUCT CATEGORIES (FULLY AUTOMATED) ===")
print()

# Get distinct category names
distinct_categories = products['product_category_name'].dropna().unique()
print(f"Found {len(distinct_categories)} distinct product categories to␣
 ↪translate")
print()

# Create translation lookup dictionary
translation_dict = {}

# Fully automated translation with deep-translator
print("Starting fully automated translation...")

# Initialize translator
translator = GoogleTranslator(source='pt', target='en')

# Translate all categories automatically
for i, category in enumerate(distinct_categories):
    # Convert snake_case to readable format
    readable_category = category.replace('_', ' ').title()

    # Translate using deep-translator
    english_name = translator.translate(readable_category)

    # Store in lookup dictionary
    translation_dict[category] = english_name

    # Print progress
    print(f"{i+1:2d}. {category} → {english_name}")

    # Small delay to avoid rate limits
    time.sleep(0.5)

print()
print(f"Automated translation complete! Processed {len(distinct_categories)}␣
 ↪categories")
print()
```

```
# Apply translation to create new column
products['product_category_name_en'] = products['product_category_name'].
 ↪map(translation_dict)

# Show sample of translations
print("Sample of translated categories:")
sample_categories = products[['product_category_name',␣
 ↪'product_category_name_en']].drop_duplicates().head(10)
for _, row in sample_categories.iterrows():
    print(f"  {row['product_category_name']} →␣
 ↪{row['product_category_name_en']}")
```

```
=== TRANSLATING PRODUCT CATEGORIES (FULLY AUTOMATED) ===

Found 74 distinct product categories to translate

Starting fully automated translation…
 1. perfumaria → Perfumery
 2. artes → Arts
 3. esporte_lazer → Sport leisure
 4. bebes → Babies
 5. utilidades_domesticas → Domestic utilities
 6. instrumentos_musicais → Musical instruments
 7. cool_stuff → Cool Stuff
 8. moveis_decoracao → Furniture Decoration
 9. eletrodomesticos → Appliances
10. brinquedos → Toys
11. cama_mesa_banho → Bath table bath table
12. construcao_ferramentas_seguranca → Construction Security Tools
13. informatica_acessorios → Computer Accessories
14. beleza_saude → HEALTH BEAUTY
15. malas_acessorios → Bags Accessories
16. ferramentas_jardim → Garden tools
17. moveis_escritorio → Furniture office
18. automotivo → Automotive
19. eletronicos → Electronics
20. fashion_calcados → Fashion Calcados
21. telefonia → Telephony
22. papelaria → Stationery shop
23. fashion_bolsas_e_acessorios → Fashion Bags and Accessories
24. pcs → PCs
25. casa_construcao → Casa Construcao
26. relogios_presentes → Watches present
27. construcao_ferramentas_construcao → Construction Tools Construction
28. pet_shop → Pet shop
29. eletroportateis → Electrophel
30. agro_industria_e_comercio → Agro Industria e Comercio
```

31. unknown_category → Unknown category
32. moveis_sala → Furniture Room
33. sinalizacao_e_seguranca → SIGNALIZATION AND SAFETY
34. climatizacao → Climatization
35. consoles_games → Games consoles
36. livros_interesse_geral → General Interest Books
37. construcao_ferramentas_ferramentas → Construction Tools Tools
38. fashion_underwear_e_moda_praia → Fashion Underwear and Beach Fashion
39. fashion_roupa_masculina → Fashion Men's Clothing
40. moveis_cozinha_area_de_servico_jantar_e_jardim → Furniture Kitchen Service Area Dinner and Garden
41. industria_comercio_e_negocios → Industry Commerce and Business
42. telefonia_fixa → Fixed telephony
43. construcao_ferramentas_iluminacao → Construction Tools Illumination
44. livros_tecnicos → Technical Books
45. eletrodomesticos_2 → ELECOMES 2
46. artigos_de_festas → Party articles
47. bebidas → Beverages
48. market_place → Market Place
49. la_cuisine → La Cuisine
50. construcao_ferramentas_jardim → Construction Tools Garden
51. fashion_roupa_feminina → Fashion Women's Clothing
52. casa_conforto → House comfort
53. audio → Audio
54. alimentos_bebidas → Drink foods
55. musica → Music
56. alimentos → Food
57. tablets_impressao_imagem → IMAGE IMPORT TABLETS
58. livros_importados → Imported books
59. portateis_casa_forno_e_cafe → Oven and coffee house portable
60. fashion_esporte → Fashion Sport
61. artigos_de_natal → Christmas articles
62. fashion_roupa_infanto_juvenil → Fashion Children's Clothing
63. dvds_blu_ray → Blu Ray DVDs
64. artes_e_artesanato → Arts and Crafts
65. pc_gamer → PC Gamer
66. moveis_quarto → FOOD FURNITURE
67. cine_foto → Cine photo
68. fraldas_higiene → Hygiene diapers
69. flores → Flower
70. casa_conforto_2 → House Comfort 2
71. portateis_cozinha_e_preparadores_de_alimentos → Kitchen Portable and Food Preparers
72. seguros_e_servicos → Insurance and services
73. moveis_colchao_e_estofado → CITTE AND UPHACE FURNITURE
74. cds_dvds_musicais → CDs Musical Dvds

Automated translation complete! Processed 74 categories

```
Sample of translated categories:
  perfumaria → Perfumery
  artes → Arts
  esporte_lazer → Sport leisure
  bebes → Babies
  utilidades_domesticas → Domestic utilities
  instrumentos_musicais → Musical instruments
  cool_stuff → Cool Stuff
  moveis_decoracao → Furniture Decoration
  eletrodomesticos → Appliances
  brinquedos → Toys
```

1.5 3. Create Product Aliases since the table doesn't have product names and the ID is not human readable

The product alias will be product_category_name_en concatenated with the rank number of the item in the category based on how many of that specific product have been ordered. This will make the reporting human readable

[5]:
```
# Prepare order_items count per product
product_item_counts = (
    order_items.groupby('product_id')
    .size()
    .reset_index(name='item_cnt')
)

# Merge item_cnt into products for ranking
products_with_cnt = products.merge(product_item_counts, on='product_id',␣
 ↪how='left')

# Fill NaN item_cnt with 0 for products with no sales
products_with_cnt['item_cnt'] = products_with_cnt['item_cnt'].fillna(0)

# For tiebreaker, use all columns except product_id and product_category_name_en
tiebreaker_cols = [col for col in products.columns if col not in ['product_id',␣
 ↪'product_category_name_en', 'product_category_name']]

# Sort and rank within each category
products_with_cnt['product_category_name_en'] =␣
 ↪products_with_cnt['product_category_name_en'].fillna('Unknown')

# Sort for ranking: by category, then item_cnt desc, then tiebreakers, then␣
 ↪product_id
sort_cols = ['product_category_name_en', 'item_cnt'] + tiebreaker_cols +␣
 ↪['product_id']
ascending = [True, False] + [True] * (len(sort_cols) - 2)
```

```
products_with_cnt = products_with_cnt.sort_values(sort_cols,␣
 ↪ascending=ascending)

# Assign rank within each category
products_with_cnt['item_rank'] = (
    products_with_cnt.groupby('product_category_name_en')
    .cumcount() + 1
)

# Create product_alias: "<product_category_name_en> item #<rank>"
products_with_cnt['product_alias'] = (
    products_with_cnt['product_category_name_en'] + ' item #' +␣
 ↪products_with_cnt['item_rank'].astype(str)
)

# Assign product_alias directly to products DataFrame by mapping
products['product_alias'] = products.set_index('product_id').index.map(
    products_with_cnt.set_index('product_id')['product_alias']
)

print(products[['product_id', 'product_alias']].head(10))
```

	product_id	product_alias
0	1e9e8ef04dbcff4541ed26657ea517e5	Perfumery item #582
1	3aa071139cb16b67ca9e5dea641aaa2f	Arts item #31
2	96bd76ec8810374ed1b65e291975717f	Sport leisure item #1958
3	cef67bcfe19066a932b7673e239eb23d	Babies item #461
4	9dc1a7de274444849c219cff195d0b71	Domestic utilities item #1310
5	41d3672d4792049fa1779bb35283ed13	Musical instruments item #278
6	732bd381ad09e530fe0a5f457d81becb	Cool Stuff item #375
7	2548af3e6e77a690cf3eb6368e9ab61e	Furniture Decoration item #139
8	37cc742be07708b53a98702e77a21a02	Appliances item #253
9	8c92109888e8cdf9d66dc7e463025574	Toys item #789

1.6 4. Create a SQLite database view to answer SQL Questions

We use in-memory SQLite and push DataFrames as tables. SQLite date ops are via `strftime`.

[6]:
```
con = sqlite3.connect(':memory:')

orders.to_sql('orders', con, index=False, if_exists='replace')
order_items.to_sql('order_items', con, index=False, if_exists='replace')
products.to_sql('products', con, index=False, if_exists='replace')
customers.to_sql('customers', con, index=False, if_exists='replace')

# indices to speed joins
with con:
```

```
    con.execute('CREATE INDEX IF NOT EXISTS idx_orders_id ON orders(order_id);')
    con.execute('CREATE INDEX IF NOT EXISTS idx_orders_customer ON␣
 ↪orders(customer_id);')
    con.execute('CREATE INDEX IF NOT EXISTS idx_oi_order ON␣
 ↪order_items(order_id);')
    con.execute('CREATE INDEX IF NOT EXISTS idx_oi_product ON␣
 ↪order_items(product_id);')
    con.execute('CREATE INDEX IF NOT EXISTS idx_oi_seller ON␣
 ↪order_items(seller_id);')
    con.execute('CREATE INDEX IF NOT EXISTS idx_prod_id ON products(product_id);
 ↪')
    con.execute('CREATE INDEX IF NOT EXISTS idx_cust_id ON␣
 ↪customers(customer_id);')

print("SQLite is ready. Tables:", pd.read_sql("SELECT name FROM sqlite_master␣
 ↪WHERE type='table'", con))
```

```
SQLite is ready. Tables:           name
0       orders
1  order_items
2     products
3    customers
```

1.6.1 SQL Tasks

Testing SQL Database Setup

[7]:
```
joined_sql = '''
    SELECT
        o.order_id,
        o.customer_id,
        o.order_status,
        o.order_purchase_timestamp,
        c.customer_unique_id,
        c.customer_city,
        c.customer_state,
        oi.order_item_id,
        oi.product_id,
        oi.seller_id,
        oi.price,
        oi.freight_value,
        p.product_category_name
    FROM
        order_items oi
        INNER JOIN orders o
            ON o.order_id = oi.order_id
        INNER JOIN products p
```

```
            ON p.product_id = oi.product_id
        INNER JOIN customers c
            ON c.customer_id = o.customer_id
'''
joined_df = pd.read_sql(joined_sql, con)
joined_df['order_date'] = pd.to_datetime(joined_df['order_purchase_timestamp']).
 ↪dt.date
joined_df['order_month'] = pd.
 ↪to_datetime(joined_df['order_purchase_timestamp']).dt.to_period('M').dt.
 ↪to_timestamp()
print(joined_df.head())
print(joined_df.shape)
```

```
                          order_id                       customer_id
order_status order_purchase_timestamp  \
0  00010242fe8c5a6d1ba2dd792cb16214  3ce436f183e68e07877b285a838db11a
delivered      2017-09-13 08:59:00
1  00018f77f2f0320c557190d7a144bdd3  f6dd3ec061db4e3987629fe6b26e5cce
delivered      2017-04-26 10:53:00
2  000229ec398224ef6ca0657da4fc703e  6489ae5e4333f3693df5ad4372dab6d3
delivered      2018-01-14 14:33:00
3  00024acbcdf0a6daa1e931b038114c75  d4eb9395c8c0431ee92fce09860c5a06
delivered      2018-08-08 10:00:00
4  00042b26cf59d7ce69dfabb4e55b4fd9  58dbd0b2d70206bf40e62cd34e84d795
delivered      2017-02-04 13:57:00

                 customer_unique_id          customer_city customer_state
order_item_id  \
0  871766c5855e863f6eccc05f988b23cb  Campos Dos Goytacazes             RJ
1
1  eb28e67c4c0b83846050ddfb8a35d051       Santa Fe Do Sul              SP
1
2  3818d81c6709e39d06b2738a8d3a2474         Para De Minas              MG
1
3  af861d436cfc08b2c2ddefd0ba074622               Atibaia              SP
1
4  64b576fb70d441e8f1b2d7d446e483c5       Varzea Paulista              SP
1

                         product_id                         seller_id   price
freight_value product_category_name  \
0  4244733e06e7ecb4970a6e2683c13e61  48436dade18ac8b2bce089ec2a041202   58.90
13.29            cool_stuff
1  e5f2d52b802189ee658865ca93d83a8f  dd7ddc04e1b6c2c614352b383efe2d36  239.90
19.93              pet_shop
2  c777355d18b72b67abbeef9df44fd0fd  5b51032eddd242adc84c38acab88f23d  199.00
17.87      moveis_decoracao
```

```
3  7634da152a4610f1595efa32f14722fc  9d7a1d34a5052409006425275ba1c2b4   12.99
12.79            perfumaria
4  ac6c3623068f30de03045865e4e10089  df560393f3a51e74553ab94004ba5c87  199.90
18.14    ferramentas_jardim

   order_date order_month
0  2017-09-13  2017-09-01
1  2017-04-26  2017-04-01
2  2018-01-14  2018-01-01
3  2018-08-08  2018-08-01
4  2017-02-04  2017-02-01
(112650, 15)
```

1.6.2 Status distribution (orders)

[8]:
```
status_counts = pd.read_sql("SELECT order_status, COUNT(*) AS n FROM orders␣
 ↪GROUP BY order_status ORDER BY n DESC", con)
fig = px.bar(status_counts, x='order_status', y='n', title='Order Status␣
 ↪Distribution', text='n')
fig.update_layout(xaxis_title='Order Status', yaxis_title='Count', bargap=0.3)
fig.show()
status_counts
```

[8]:

	order_status	n
0	delivered	96478
1	shipped	1107
2	canceled	625
3	unavailable	609
4	invoiced	314
5	processing	301
6	created	5
7	approved	2

1.7 4. SQL Task 1 — Top-selling products (overall & by region)

Definition: Sales = sum of `price`, not including freight values. Gross (GMV) = `price` + `freight`. Item Count = The quantity of the item that was ordered. Assuming top-selling is based on item value only, and does not include freight value.

Assumptions: The definition of "top-selling" is based on sales revenue, i.e. the item(s) that has the highest total dollar value sold, excluding freight.

Only include the following order statuses: - delivered - shipped - invoiced - processing - created - approved

Overall (by product_id):

[9]:
```
sql_top_products_overall = '''
WITH
```

```
base AS (
  SELECT
    p.product_alias,
    MAX(p.product_category_name_en) AS product_category_name,
    SUM(oi.price) AS sales,
    SUM(oi.price + IFNULL(oi.freight_value,0)) AS gross,
    COUNT(*) AS item_cnt
  FROM
    order_items oi
    JOIN orders o
      ON o.order_id = oi.order_id
    JOIN products p
      ON p.product_id = oi.product_id
  WHERE TRUE
    AND o.order_status IN ('delivered', 'shipped', 'invoiced', 'processing',␣
 ↪'created', 'approved')
  GROUP BY p.product_alias
),

ranked AS (
  SELECT
    product_alias,
    product_category_name,
    sales,
    gross,
    item_cnt,
    RANK() OVER (ORDER BY sales DESC) AS rnk
  FROM base
)

SELECT
  *
FROM
  ranked
WHERE TRUE
  AND rnk <= 15
ORDER BY rnk;
'''
top_products_overall = pd.read_sql(sql_top_products_overall, con)

# Format columns for hover
top_products_overall['sales_fmt'] = top_products_overall['sales'].apply(lambda␣
 ↪x: f"${x:,.2f}")
top_products_overall['gross_fmt'] = top_products_overall['gross'].apply(lambda␣
 ↪x: f"${x:,.2f}")
```

```
top_products_overall['item_cnt_fmt'] = top_products_overall['item_cnt'].
 ↪apply(lambda x: f"{x:,}")

fig = px.bar(
    top_products_overall,
    x='product_alias',
    y='sales',
    title='Top-Selling Products ($)',
    hover_data={
        'sales_fmt': True,
        'gross_fmt': True,
        'item_cnt_fmt': True,
        'sales': False,
        'gross': False,
        'item_cnt': False,
        'product_alias': True
    },
    labels={'sales_fmt': 'Sales', 'gross_fmt': 'Gross', 'item_cnt_fmt': 'Item␣
 ↪Count'}
)

# Add sales amount as text on bars
fig.update_traces(
    text=top_products_overall['sales'].apply(lambda x: f"${x:,.2f}"),
    textposition='outside'
)

fig.update_layout(
    xaxis_title='Product Name Alias',
    yaxis_title='Total Sales ($)',
    uniformtext_minsize=8,
    uniformtext_mode='hide',
    height=600,
    margin=dict(t=80, b=80, l=80, r=40)
)

fig.show()
top_products_overall.head(20)
```

[9]:

```
                    product_alias  product_category_name     sales     gross
item_cnt  rnk   sales_fmt   gross_fmt  \
0           HEALTH BEAUTY item #4          HEALTH BEAUTY  63885.00  67606.10
195    1  $63,885.00  $67,606.10
1           HEALTH BEAUTY item #6          HEALTH BEAUTY  54730.20  59093.99
156    2  $54,730.20  $59,093.99
2                    PCs item #1                    PCs  48899.34  50326.18
35     3  $48,899.34  $50,326.18
```

```
3    Computer Accessories item #1   Computer Accessories  46916.51  60597.29
341    4  $46,916.51  $60,597.29
4   Bath table bath table item #1  Bath table bath table  42938.66  50963.60
487    5  $42,938.66  $50,963.60
5    Computer Accessories item #2   Computer Accessories  41082.60  48212.22
274    6  $41,082.60  $48,212.22
6                 Babies item #8                Babies  38907.32  40311.95
38    7  $38,907.32  $40,311.95
7             Cool Stuff item #8            Cool Stuff  37733.90  41725.81
63    8  $37,733.90  $41,725.81
8         Watches present item #1        Watches present  37683.42  39957.93
323    9  $37,683.42  $39,957.93
9    Furniture Decoration item #1   Furniture Decoration  37608.90  44820.76
527   10  $37,608.90  $44,820.76
10        Watches present item #3        Watches present  31786.82  35344.09
194   11  $31,786.82  $35,344.09
11        Watches present item #7        Watches present  31623.81  33607.57
123   12  $31,623.81  $33,607.57
12        Watches present item #6        Watches present  30467.50  32790.74
143   13  $30,467.50  $32,790.74
13  Bath table bath table item #2  Bath table bath table  29997.36  33316.76
154   14  $29,997.36  $33,316.76
14       Watches present item #15        Watches present  29024.48  30449.69
45   15  $29,024.48  $30,449.69

   item_cnt_fmt
0           195
1           156
2            35
3           341
4           487
5           274
6            38
7            63
8           323
9           527
10          194
11          123
12          143
13          154
14           45
```

By Region (state). We aggregate by `customer_state` then rank within each state.

[10]:
```
# Top products by region - using product_alias instead of product_id
print("=== TOP PRODUCTS BY REGION ===")
print()
```

```
# Get all unique regions (states)
regions = pd.read_sql("SELECT DISTINCT customer_state FROM customers ORDER BY␣
 ↪customer_state", con)['customer_state'].tolist()
print(f"Found {len(regions)} regions: {regions}")
print()

# SQL query for top products by region
sql_top_products_by_region = '''
WITH base AS (
  SELECT
    c.customer_state,
    oi.product_id,
    MAX(p.product_alias) AS product_alias,
    MAX(p.product_category_name_en) AS product_category_name,
    SUM(oi.price) AS sales,
    SUM(oi.price + IFNULL(oi.freight_value,0)) AS gross,
    COUNT(*) AS item_cnt
  FROM order_items oi
    JOIN orders o ON o.order_id = oi.order_id
    JOIN products p ON p.product_id = oi.product_id
    JOIN customers c ON c.customer_id = o.customer_id
  WHERE TRUE
    AND o.order_status IN ('delivered', 'shipped', 'invoiced', 'processing',␣
 ↪'created', 'approved')
  GROUP BY c.customer_state, oi.product_id
),
ranked AS (
  SELECT
    customer_state,
    product_id,
    product_alias,
    product_category_name,
    sales,
    gross,
    item_cnt,
    RANK() OVER (PARTITION BY customer_state ORDER BY sales DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10 ORDER BY customer_state, rnk;
'''

top_products_by_region = pd.read_sql(sql_top_products_by_region, con)
top_products_by_region.head(20)
```

```
=== TOP PRODUCTS BY REGION ===
```

```
Found 27 regions: ['AC', 'AL', 'AM', 'AP', 'BA', 'CE', 'DF', 'ES', 'GO', 'MA',
'MG', 'MS', 'MT', 'PA', 'PB', 'PE', 'PI', 'PR', 'RJ', 'RN', 'RO', 'RR', 'RS',
'SC', 'SE', 'SP', 'TO']
```

[10]:

	customer_state	product_id	product_alias	product_category_name
0	AC	a5215a7a9f46c4185b12f38e9ddf2abc	item #5	PCs
1	AC	cf8587a915960e2a8b57c5db574239ad	item #383	Watches present
2	AC	6767719f80aabbbf16ab2491899c32d9	item #223	Telephony
3	AC	fe59a1e006df3ac42bf0ceb876d70969	item #123	Computer Accessories
4	AC	a082e9e8862e9af2f5a67c2dd1594010	item #696	Sport leisure
5	AC	af0a99476d96dcc1a1baa7c0d9ff6b9d	item #48	HEALTH BEAUTY
6	AC	1cc61b32763a4d816212b3507b6b6c59	item #4	General Interest Books
7	AC	d5ad5c4a843732aee4cecc1bbbf276dc	item #475	Garden tools
8	AC	b81a05d0dd312ece2140846909f5ef81	item #223	Furniture Decoration
9	AC	3dacb3ae011b40803a508b23392e15a0	item #217	Stationery shop
10	AL	6cdd53843498f92890544667809f1595	item #6	HEALTH BEAUTY
11	AL	7ea73eb6e7e6ebb1113ba90f839b3402	item #809	Computer Accessories
12	AL	3976c6466748442470251341eb6f92a3	item #387	Pet shop
13	AL	606b5bf4a5c5ac2f72196afa18b77d3d	item #336	Watches present
14	AL	76951acb34204078c0dc377dfc4233aa	item #944	HEALTH BEAUTY
15	AL	d8655d4a43e4a9ac85575a5f9ef816b0	item #66	ELECOMES 2
16	AL	d5dbb4d9ecbbf2e312169e4c8f1b57f0	item #35	Agro Industria e Comercio
17	AL	fb01a5fc09b9b9563c2ee41a22f07d54	item #5	Games consoles
18	AL	f819f0c84a64f02d3a5606ca95edd272	item #15	Watches present
19	AL	d6160fb7873f184099d9bc95e30376af	item #1	PCs

	sales	gross	item_cnt	rnk
0	1200.00	1251.70	1	1
1	961.60	995.18	1	2
2	839.99	905.93	1	3
3	809.10	861.26	1	4
4	549.00	618.60	1	5
5	527.90	591.88	1	6
6	524.90	595.49	1	7
7	510.00	548.93	1	8
8	479.40	646.44	6	9
9	399.00	448.30	1	10
10	3163.20	3500.92	9	1
11	2159.98	2269.98	2	2
12	1798.00	1942.00	1	3
13	1597.80	1653.98	2	4
14	1597.35	1650.56	1	5
15	1595.00	1658.81	1	6
16	1476.30	1518.55	1	7
17	1379.78	1477.14	2	8
18	1368.90	1453.58	2	9
19	1200.00	1227.89	1	10

1.8 4. SQL Task 2 — Most popular categories

Defining popularity in 3 ways: **(1)** by **item count**, **(2)** by **order count**, and by (3) **Category GMV**

[11]:
```
sql_popular_categories = '''
WITH base AS (
  SELECT
    p.product_category_name_en AS product_category_name,
    COUNT(*) AS item_cnt,
    SUM(oi.price) AS sales,
    SUM(oi.price + IFNULL(oi.freight_value,0)) AS gross
  FROM order_items oi
    JOIN orders o ON o.order_id = oi.order_id
    JOIN products p ON p.product_id = oi.product_id
  WHERE TRUE
    AND o.order_status IN ('delivered', 'shipped', 'invoiced', 'processing',␣
 ↪'created', 'approved')
  GROUP BY p.product_category_name_en
),

-- Creating ranks for each category based on the different metrics.  Using␣
 ↪RANK() instead of ROW_NUMBER() because RANK() allows for ties.
rank_cnt AS (
```

```
  SELECT *, RANK() OVER (ORDER BY item_cnt DESC) AS r_cnt FROM base
),
rank_sales AS (
  SELECT *, RANK() OVER (ORDER BY sales DESC) AS r_gmv FROM base
),
rank_gross AS (
  SELECT *, RANK() OVER (ORDER BY gross DESC) AS r_gross FROM base
)

SELECT
  b.product_category_name,
  b.item_cnt,
  b.sales,
  b.gross,
  rc.r_cnt,
  rs.r_gmv,
  rg.r_gross
FROM base b
  JOIN rank_cnt rc USING (product_category_name, item_cnt, sales, gross)
  JOIN rank_sales rs USING (product_category_name, item_cnt, sales, gross)
  JOIN rank_gross rg USING (product_category_name, item_cnt, sales, gross)
WHERE TRUE
  AND (
    rc.r_cnt <= 15
    OR
    rs.r_gmv <= 15
    OR
    rg.r_gross <= 15
  )
ORDER BY r_gmv, r_cnt, r_gross;
'''
popular_categories = pd.read_sql(sql_popular_categories, con)

sorted_popular_categories = popular_categories.sort_values('item_cnt',␣
 ↪ascending=False)
fig1 = px.bar(sorted_popular_categories, x='product_category_name',␣
 ↪y='item_cnt',
              title='Most Popular Categories by Item Count',␣
 ↪hover_data=['sales','gross'])
fig1.update_layout(xaxis_title='Category', yaxis_title='Item Count')
fig1.show()

fig2 = px.bar(popular_categories, x='product_category_name', y='sales',
              title='Most Popular Categories by Sales Revenue',␣
 ↪hover_data=['item_cnt','gross'])
fig2.update_layout(xaxis_title='Category', yaxis_title='GMV')
fig2.show()
```

```
fig3 = px.bar(popular_categories, x='product_category_name', y='gross',
              title='Most Popular Categories by GMV',␣
 ↪hover_data=['item_cnt','sales'])
fig3.update_layout(xaxis_title='Category', yaxis_title='GMV')
fig3.show()

popular_categories.head(20)
```

[11]:

	product_category_name	item_cnt	sales	gross	r_cnt	r_gmv	r_gross
0	HEALTH BEAUTY	9634	1255695.13	1437665.78	2	1	1
1	Watches present	5970	1198185.21	1298292.47	7	2	2
2	Bath table bath table	11097	1035964.06	1240386.13	1	3	3
3	Sport leisure	8590	979740.92	1147244.63	3	4	4
4	Computer Accessories	7781	904322.02	1050941.58	5	5	5
5	Furniture Decoration	8298	727465.05	899626.04	4	6	6
6	Domestic utilities	6915	626825.80	772035.14	6	7	7
7	Cool Stuff	3779	620770.49	704086.24	12	8	8
8	Automotive	4204	586585.73	678606.64	10	9	9
9	Garden tools	4328	481009.94	579525.20	9	10	10
10	Toys	4083	479808.54	556783.29	11	11	11
11	Babies	3043	410312.20	478320.00	14	12	12
12	Perfumery	3402	396599.31	450580.47	13	13	13
13	Telephony	4527	322342.64	393306.02	8	14	14
14	Furniture office	1690	273580.70	342073.02	19	15	15
15	Electronics	2755	157079.54	203416.77	15	22	22

1.9 5. SQL Task 3 — Monthly, Quarterly, Yearly sales (all products)

We aggregate GMV over time and split by status.

[12]:
```
sql_time_sales = '''
WITH

base AS (
  SELECT
    DATE(o.order_purchase_timestamp) AS d,
    STRFTIME('%Y-%m', o.order_purchase_timestamp) AS ym,
    CAST(STRFTIME('%Y', o.order_purchase_timestamp) AS INT) AS y,
    CAST(STRFTIME('%m', o.order_purchase_timestamp) AS INT) AS m,
    CASE
      WHEN CAST(STRFTIME('%m', o.order_purchase_timestamp) AS INT) IN (1,2,3)␣
 ↪THEN 1
      WHEN CAST(STRFTIME('%m', o.order_purchase_timestamp) AS INT) IN (4,5,6)␣
 ↪THEN 2
      WHEN CAST(STRFTIME('%m', o.order_purchase_timestamp) AS INT) IN (7,8,9)␣
 ↪THEN 3
      WHEN CAST(STRFTIME('%m', o.order_purchase_timestamp) AS INT) IN␣
 ↪(10,11,12) THEN 4
    END AS q,
    SUM(oi.price) AS sales,
    SUM(oi.price + IFNULL(oi.freight_value,0)) AS gross
  FROM
    order_items oi
    JOIN orders o
      ON o.order_id = oi.order_id
  WHERE TRUE
    AND o.order_status IN ('delivered', 'shipped', 'invoiced', 'processing',␣
 ↪'created', 'approved')
  GROUP BY o.order_status, d
),

monthly AS (
  SELECT
      ym,
      SUM(sales) AS sales,
      SUM(gross) AS gross
  FROM base GROUP BY ym
),

quarterly AS (
  SELECT
    y,
    q,
    SUM(sales) AS sales,
    SUM(gross) AS gross
  FROM
    base
```

```
  GROUP BY y, q
),

yearly AS (
  SELECT
    y,
    SUM(sales) AS sales,
    SUM(gross) AS gross
  FROM
    base GROUP BY y
)

SELECT
  'monthly' AS time_grain,
  ym AS period,
  sales,
  gross
FROM
  monthly

UNION ALL

SELECT
  'quarterly' AS time_grain,
  CAST(y AS TEXT)||'-Q'||CAST(q AS TEXT) AS period,
  sales,
  gross
FROM
  quarterly

UNION ALL

SELECT
  'yearly' AS time_grain,
  CAST(y AS TEXT) AS period,
  sales,
  gross
FROM
  yearly
ORDER BY time_grain, period;
'''
time_sales = pd.read_sql(sql_time_sales, con)
time_sales
```

[12]:

	time_grain	period	sales	gross
0	monthly	2016-09	207.86	279.69
1	monthly	2016-10	44507.30	51354.52

2	monthly	2016-12	10.90	19.62
3	monthly	2017-01	120098.27	136943.46
4	monthly	2017-02	244959.35	283561.69
5	monthly	2017-03	368341.32	425617.96
6	monthly	2017-04	353842.98	405848.61
7	monthly	2017-05	503159.19	582710.83
8	monthly	2017-06	429916.61	499652.24
9	monthly	2017-07	492287.30	578753.73
10	monthly	2017-08	568245.79	661903.52
11	monthly	2017-09	621415.91	717102.72
12	monthly	2017-10	660179.62	764756.03
13	monthly	2017-11	1003862.14	1172191.68
14	monthly	2017-12	742183.79	861526.77
15	monthly	2018-01	945456.29	1101920.01
16	monthly	2018-02	837895.43	979486.16
17	monthly	2018-03	981051.06	1152656.99
18	monthly	2018-04	993592.98	1156248.89
19	monthly	2018-05	992871.75	1145686.46
20	monthly	2018-06	863265.53	1020381.90
21	monthly	2018-07	878044.27	1039783.58
22	monthly	2018-08	848860.10	996973.51
23	monthly	2018-09	145.00	166.46
24	quarterly	2016-Q3	207.86	279.69
25	quarterly	2016-Q4	44518.20	51374.14
26	quarterly	2017-Q1	733398.94	846123.11
27	quarterly	2017-Q2	1286918.78	1488211.68
28	quarterly	2017-Q3	1681949.00	1957759.97
29	quarterly	2017-Q4	2406225.55	2798474.48
30	quarterly	2018-Q1	2764402.78	3234063.16
31	quarterly	2018-Q2	2849730.26	3322317.25
32	quarterly	2018-Q3	1727049.37	2036923.55
33	yearly	2016	44726.06	51653.83
34	yearly	2017	6108492.27	7090569.24
35	yearly	2018	7341182.41	8593303.96

[13]:
```
monthly = time_sales[time_sales['time_grain']=='monthly'].copy()
fig_m = px.line(monthly, x='period', y='sales',
                title='Monthly Sales', markers=True)
fig_m.update_layout(xaxis_title='Month', yaxis_title='sales')
fig_m.show()

quarterly = time_sales[time_sales['time_grain']=='quarterly'].copy()
fig_q = px.line(quarterly, x='period', y='sales',
                title='Quarterly Sales', markers=True)
fig_q.update_layout(xaxis_title='Quarter', yaxis_title='sales')
fig_q.show()
```

```
yearly = time_sales[time_sales['time_grain']=='yearly'].copy()
fig_y = px.bar(yearly, x='period', y='sales',
               title='Yearly Sales', text='sales')
fig_y.update_layout(xaxis_title='Year', yaxis_title='sales')
fig_y.show()
```

[ ]:

[ ]:

1.10 6. SQL Task 4 — Average sale by category & top category by location

Two related metrics:

- **AOV by Category**: For each (order_id, category), sum item prices then average across orders.
- **Avg Item Price by Category**: Simpler item-level mean price.
- **Top Category by State**: For each state + status, the category with max GMV.

1.10.1 AOV by Category:

[14]:
```
# AOV by Category (order-level aggregation by category)
sql_aov_by_cat = '''
WITH

order_cat AS (
  SELECT
    o.order_id,
    p.product_category_name_en,
    SUM(oi.price) AS order_cat_value
  FROM
    order_items oi
    JOIN orders o
      ON o.order_id = oi.order_id
    JOIN products p
      ON p.product_id = oi.product_id
  WHERE TRUE
    AND o.order_status IN ('delivered', 'shipped', 'invoiced', 'processing',␣
 ↪'created', 'approved')
  GROUP BY o.order_id, p.product_category_name_en
)

SELECT
  product_category_name_en,
  AVG(order_cat_value) AS aov_by_category
FROM order_cat
GROUP BY product_category_name_en
ORDER BY aov_by_category DESC;
```

```
'''
aov_by_cat = pd.read_sql(sql_aov_by_cat, con)

fig_aov = px.bar(aov_by_cat, x='product_category_name_en', y='aov_by_category',
                 title='Average Order Value (by Category & Status)')
fig_aov.update_layout(xaxis_title='Category', yaxis_title='AOV (sales per␣
 ↪order-category)')
fig_aov.show()
```

1.10.2 Top Category by Customer State:

[15]:
```
# Top category by state (max sales)
sql_top_cat_by_state = '''
WITH base AS (
  SELECT
    c.customer_state,
    p.product_category_name_en,
    SUM(oi.price) AS sales
  FROM
    order_items oi
    JOIN orders o
      ON o.order_id = oi.order_id
    JOIN customers c
      ON c.customer_id = o.customer_id
    JOIN products p
      ON p.product_id = oi.product_id
  WHERE TRUE
    AND o.order_status IN ('delivered', 'shipped', 'invoiced', 'processing',␣
 ↪'created', 'approved')
  GROUP BY c.customer_state, p.product_category_name_en
),
ranked AS (
  SELECT
    customer_state,
    product_category_name_en,
    sales,
    RANK() OVER (PARTITION BY customer_state ORDER BY sales DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk = 1 ORDER BY customer_state;
'''
top_cat_by_state = pd.read_sql(sql_top_cat_by_state, con)
fig_top_cat = px.bar(top_cat_by_state, x='customer_state', y='sales' if 'sales'␣
 ↪in top_cat_by_state.columns else 'sales',
                     color='product_category_name_en',
                     title='Top Category by State (Max Sales)',
```

```
                    labels={'customer_state': 'State', 'sales': 'Sales',␣
 ↪'sales': 'Sales', 'product_category_name_en': 'Top Category'})
fig_top_cat.update_layout(xaxis_title='State', yaxis_title='Sales ($)',␣
 ↪legend_title='Top Category')
fig_top_cat.show()
```

1.11 7. Python Task 1 — Top 10 stores by highest average daily sales

Store is defined as a `seller_id`. For each seller and day, we sum GMV, then compute the **average of daily totals**. Then select the top 10 stores.

[16]:
```
# Top 10 stores with highest average daily sales
print("=== TOP 10 STORES BY AVERAGE DAILY SALES ===")
print()

# Filter orders by status
valid_statuses = ['delivered', 'shipped', 'invoiced', 'processing', 'created',␣
 ↪'approved']
filtered_orders = orders[orders['order_status'].isin(valid_statuses)].copy()

print(f"Filtered orders to {len(filtered_orders)} orders with valid statuses")
print(f"Order statuses included: {valid_statuses}")
print()

# Join orders with order_items to get seller and sales data
daily_sales = (filtered_orders
    .merge(order_items, on='order_id', how='inner')
    .assign(
        order_date=pd.to_datetime(filtered_orders['order_purchase_timestamp']).
 ↪dt.date
    )
    .groupby(['seller_id', 'order_date'], as_index=False)
    .agg(daily_sales=('price', 'sum'))
)

print(f"Daily sales data: {len(daily_sales)} seller-date combinations")
print()

# Calculate average daily sales per seller
avg_daily_sales = (daily_sales
    .groupby('seller_id', as_index=False)
    .agg(
        avg_daily_sales=('daily_sales', 'mean'),
        total_days=('order_date', 'nunique'),
        total_sales=('daily_sales', 'sum')
    )
    .sort_values('avg_daily_sales', ascending=False)
```

```
)

print(f"Calculated average daily sales for {len(avg_daily_sales)} sellers")
print()

# Get top 10 stores
top_10_stores = avg_daily_sales.head(10).copy()

# Format for display
top_10_stores['avg_daily_sales_fmt'] = top_10_stores['avg_daily_sales'].
 ↪apply(lambda x: f"${x:,.2f}")
top_10_stores['total_sales_fmt'] = top_10_stores['total_sales'].apply(lambda x:␣
 ↪f"${x:,.2f}")

print("Top 10 stores by average daily sales:")
for i, (_, row) in enumerate(top_10_stores.iterrows(), 1):
    print(f"{i:2d}. {row['seller_id'][:8]}... - Avg Daily:␣
 ↪{row['avg_daily_sales_fmt']}, "
          f"Total Sales: {row['total_sales_fmt']}, Active Days:␣
 ↪{row['total_days']}")
print()

# Create bar chart
fig = px.bar(
    top_10_stores,
    x='seller_id',
    y='avg_daily_sales',
    title='Top 10 Stores by Average Daily Sales',
    hover_data={
        'avg_daily_sales_fmt': True,
        'total_sales_fmt': True,
        'total_days': True,
        'avg_daily_sales': False,
        'total_sales': False
    },
    labels={
        'avg_daily_sales_fmt': 'Avg Daily Sales',
        'total_sales_fmt': 'Total Sales',
        'total_days': 'Active Days'
    }
)

# Add sales amount as text on bars
fig.update_traces(
    text=top_10_stores['avg_daily_sales'].apply(lambda x: f"${x:,.2f}"),
    textposition='outside'
)
```

```
fig.update_layout(
    xaxis_title='Seller ID',
    yaxis_title='Average Daily Sales ($)',
    uniformtext_minsize=8,
    uniformtext_mode='hide',
    height=600,
    margin=dict(t=80, b=80, l=80, r=40)
)

fig.show()

# Display the data table
print("Top 10 stores data:")
display(top_10_stores[['seller_id', 'avg_daily_sales', 'total_sales',␣
 ↪'total_days']].head(10))
```

```
=== TOP 10 STORES BY AVERAGE DAILY SALES ===

Filtered orders to 98207 orders with valid statuses
Order statuses included: ['delivered', 'shipped', 'invoiced', 'processing',
'created', 'approved']

Daily sales data: 76418 seller-date combinations

Calculated average daily sales for 2980 sellers

Top 10 stores by average daily sales:
 1. 80ceebb4… - Avg Daily: $6,729.00, Total Sales: $6,729.00, Active Days: 1
 2. ee27a8f1… - Avg Daily: $6,499.00, Total Sales: $6,499.00, Active Days: 1
 3. 6fa9202c… - Avg Daily: $4,676.16, Total Sales: $4,676.16, Active Days: 1
 4. 585175ec… - Avg Daily: $3,549.00, Total Sales: $3,549.00, Active Days: 1
 5. abe021b0… - Avg Daily: $3,360.00, Total Sales: $6,720.00, Active Days: 2
 6. a00824eb… - Avg Daily: $3,133.32, Total Sales: $9,399.97, Active Days: 3
 7. e2a1ac9b… - Avg Daily: $2,999.89, Total Sales: $8,999.67, Active Days: 3
 8. e908c0f3… - Avg Daily: $2,951.00, Total Sales: $2,951.00, Active Days: 1
 9. d63c73ef… - Avg Daily: $2,799.00, Total Sales: $5,598.00, Active Days: 2
10. 1444c08e… - Avg Daily: $2,749.00, Total Sales: $2,749.00, Active Days: 1


Top 10 stores data:
                          seller_id  avg_daily_sales  total_sales  total_days
1537  80ceebb4ee9b31afb6c6a916a574a1e2      6729.000000      6729.00           1
2780  ee27a8f15b1dded4d213a468ba4eb391      6499.000000      6499.00           1
1323  6fa9202c10491e472dffd59a3e82b2a3      4676.160000      4676.16           1
1048  585175ec331ea177fa47199e39a6170a      3549.000000      3549.00           1
2009  abe021b01ba992245271b9aa422032df      3360.000000      6720.00           2
1886  a00824eb9093d40e589b940ec45c4eb0      3133.323333      9399.97           3
```

```
2634   e2a1ac9bf33e5549a2a4f834e70df2f8      2999.890000      8999.67           3
2715   e908c0f3646e8b60375734a350d95d71      2951.000000      2951.00           1
2501   d63c73efd41eb002280e7ec831424edb      2799.000000      5598.00           2
236    1444c08e64d55fb3c25f0f09c07ffcf2      2749.000000      2749.00           1
```

1.11.1 The above top 10 stores are only active for 1 - 3 days. The below analysis checks the distribution of active selling days.

The results show that higher AOV stores generally don't have many days of selling. This infers that higher AOV stores are hype-based sellers that have high store traffic in limited amounts of time, potentially stressing the scaling of Salla's infrastructure.

[17]:
```
# Distribution of active days per seller analysis
print("=== ACTIVE DAYS DISTRIBUTION PER SELLER ===")
print()

# We need to recalculate the data from the previous cell
# Filter orders by status
valid_statuses = ['delivered', 'shipped', 'invoiced', 'processing', 'created',␣
 ↪'approved']
filtered_orders = orders[orders['order_status'].isin(valid_statuses)].copy()

# Join orders with order_items to get seller and sales data
daily_sales = (filtered_orders
    .merge(order_items, on='order_id', how='inner')
    .assign(
        order_date=pd.to_datetime(filtered_orders['order_purchase_timestamp']).
 ↪dt.date
    )
    .groupby(['seller_id', 'order_date'], as_index=False)
    .agg(daily_sales=('price', 'sum'))
)

# Calculate average daily sales per seller
avg_daily_sales = (daily_sales
    .groupby('seller_id', as_index=False)
    .agg(
        avg_daily_sales=('daily_sales', 'mean'),
        total_days=('order_date', 'nunique'),
        total_sales=('daily_sales', 'sum')
    )
    .sort_values('avg_daily_sales', ascending=False)
)

# Active days statistics
active_days_stats = avg_daily_sales['total_days'].describe()
print("Active days statistics:")
print(active_days_stats)
```

```
print()

# Distribution by ranges
print("Active days distribution by ranges:")
ranges = [
    (1, 5, "1-5 days"),
    (6, 10, "6-10 days"),
    (11, 20, "11-20 days"),
    (21, 50, "21-50 days"),
    (51, 100, "51-100 days"),
    (101, 200, "101-200 days"),
    (201, float('inf'), "200+ days")
]

for min_days, max_days, label in ranges:
    if max_days == float('inf'):
        count = (avg_daily_sales['total_days'] >= min_days).sum()
    else:
        count = ((avg_daily_sales['total_days'] >= min_days) &␣
 ↪(avg_daily_sales['total_days'] <= max_days)).sum()
    percentage = (count / len(avg_daily_sales)) * 100
    print(f"  {label}: {count} sellers ({percentage:.1f}%)")

print()

# Show some examples
print("Examples of sellers by active days:")
print("Sellers with 1-5 active days (sample):")
low_activity = avg_daily_sales[avg_daily_sales['total_days'] <= 5].head(5)
for _, row in low_activity.iterrows():
    print(f"  {row['seller_id'][:8]}... - {row['total_days']} days, Avg Daily:␣
 ↪${row['avg_daily_sales']:,.2f}")

print("\nSellers with 200+ active days (sample):")
high_activity = avg_daily_sales[avg_daily_sales['total_days'] >= 200].head(5)
for _, row in high_activity.iterrows():
    print(f"  {row['seller_id'][:8]}... - {row['total_days']} days, Avg Daily:␣
 ↪${row['avg_daily_sales']:,.2f}")

print()

# Create a histogram of active days
fig_hist = px.histogram(
    avg_daily_sales,
    x='total_days',
    title='Distribution of Active Days per Seller',
    nbins=30,
```

```
    labels={'total_days': 'Active Days', 'count': 'Number of Sellers'}
)

fig_hist.update_layout(
    xaxis_title='Active Days',
    yaxis_title='Number of Sellers',
    height=500,
    margin=dict(t=80, b=80, l=80, r=40)
)

fig_hist.show()

# Additional analysis: correlation between active days and average daily sales
print("=== CORRELATION ANALYSIS ===")
print()

correlation = avg_daily_sales['total_days'].
 ↪corr(avg_daily_sales['avg_daily_sales'])
print(f"Correlation between active days and average daily sales: {correlation:.
 ↪3f}")

# Scatter plot
fig_scatter = px.scatter(
    avg_daily_sales,
    x='total_days',
    y='avg_daily_sales',
    title='Active Days vs Average Daily Sales',
    labels={'total_days': 'Active Days', 'avg_daily_sales': 'Average Daily␣
 ↪Sales ($)'},
    hover_data=['total_sales']
)

fig_scatter.update_layout(
    xaxis_title='Active Days',
    yaxis_title='Average Daily Sales ($)',
    height=500,
    margin=dict(t=80, b=80, l=80, r=40)
)

fig_scatter.show()
```

```
=== ACTIVE DAYS DISTRIBUTION PER SELLER ===

Active days statistics:
count    2980.000000
mean       25.643624
std        54.511014
```

```
min          1.000000
25%          2.000000
50%          7.000000
75%         22.000000
max        530.000000
Name: total_days, dtype: float64

Active days distribution by ranges:
  1-5 days: 1327 sellers (44.5%)
  6-10 days: 479 sellers (16.1%)
  11-20 days: 383 sellers (12.9%)
  21-50 days: 391 sellers (13.1%)
  51-100 days: 217 sellers (7.3%)
  101-200 days: 120 sellers (4.0%)
  200+ days: 63 sellers (2.1%)

Examples of sellers by active days:
Sellers with 1-5 active days (sample):
  80ceebb4… - 1 days, Avg Daily: $6,729.00
  ee27a8f1… - 1 days, Avg Daily: $6,499.00
  6fa9202c… - 1 days, Avg Daily: $4,676.16
  585175ec… - 1 days, Avg Daily: $3,549.00
  abe021b0… - 2 days, Avg Daily: $3,360.00

Sellers with 200+ active days (sample):
  53243585… - 250 days, Avg Daily: $766.26
  7e93a43e… - 217 days, Avg Daily: $691.82
  fa1c13f2… - 324 days, Avg Daily: $531.92
  4869f7a5… - 466 days, Avg Daily: $432.47
  5dceca12… - 233 days, Avg Daily: $429.63


=== CORRELATION ANALYSIS ===

Correlation between active days and average daily sales: -0.038
```

1.12 8. Python Task 2 — % monthly growth in sales per store

We compute monthly GMV per store, then `pct_change()` to get growth.

[18]:
```
# Monthly Growth in Sales by Seller
print("=== MONTHLY GROWTH IN SALES BY SELLER ===")
print()

# Use original dataframes with same status filtering as before
valid_statuses = ['delivered', 'shipped', 'invoiced', 'processing', 'created',␣
 ↪'approved']
filtered_orders = orders[orders['order_status'].isin(valid_statuses)].copy()
```

```
print(f"Using {len(filtered_orders)} orders with valid statuses: {valid_statuses}")
print()

# Create monthly sales data by seller
monthly_sales = (filtered_orders
    .merge(order_items, on='order_id', how='inner')
    .assign(
        order_ym=pd.to_datetime(filtered_orders['order_purchase_timestamp']).dt.to_period('M').dt.to_timestamp()
    )
    .groupby(['seller_id', 'order_ym'], as_index=False)
    .agg(sales=('price', 'sum'))
    .sort_values(['seller_id', 'order_ym'])
)

print(f"Monthly sales data: {len(monthly_sales)} seller-month combinations")
print()

# Calculate monthly growth by seller
monthly_sales['pct_growth'] = (monthly_sales
    .groupby('seller_id')['sales']
    .pct_change()
    .fillna(0)  # First month for each seller = 0% growth
)

print("Sample of monthly growth data:")
print(monthly_sales.head(10))
print()

# Show summary statistics
print("=== MONTHLY GROWTH SUMMARY ===")
print()

growth_stats = monthly_sales['pct_growth'].describe()
print("Growth rate statistics:")
print(growth_stats)
print()

# Count of sellers with positive vs negative growth
positive_growth = (monthly_sales['pct_growth'] > 0).sum()
negative_growth = (monthly_sales['pct_growth'] < 0).sum()
zero_growth = (monthly_sales['pct_growth'] == 0).sum()

print(f"Growth distribution:")
```

```
print(f"  Positive growth: {positive_growth} seller-months ({positive_growth/
 ↪len(monthly_sales)*100:.1f}%)")
print(f"  Negative growth: {negative_growth} seller-months ({negative_growth/
 ↪len(monthly_sales)*100:.1f}%)")
print(f"  Zero growth: {zero_growth} seller-months ({zero_growth/
 ↪len(monthly_sales)*100:.1f}%)")
print()

# Top and bottom growth rates
print("Top 10 highest growth rates:")
top_growth = monthly_sales.nlargest(10, 'pct_growth')[['seller_id', 'order_ym',␣
 ↪'sales', 'pct_growth']]
for i, (_, row) in enumerate(top_growth.iterrows(), 1):
    print(f"{i:2d}. {row['seller_id'][:8]}... - {row['order_ym'].
 ↪strftime('%Y-%m')} - "
          f"Growth: {row['pct_growth']*100:.1f}% - Sales: ${row['sales']:,.2f}")

print()
print("Top 10 lowest growth rates:")
bottom_growth = monthly_sales.nsmallest(10, 'pct_growth')[['seller_id',␣
 ↪'order_ym', 'sales', 'pct_growth']]
for i, (_, row) in enumerate(bottom_growth.iterrows(), 1):
    print(f"{i:2d}. {row['seller_id'][:8]}... - {row['order_ym'].
 ↪strftime('%Y-%m')} - "
          f"Growth: {row['pct_growth']*100:.1f}% - Sales: ${row['sales']:,.2f}")

print()

print("=== MONTHLY GROWTH DISTRIBUTION - BOXPLOT BY MONTH (LOG SCALE/CLIPPED)␣
 ↪===")
print()

# Prepare data for visualization
monthly_sales_display = monthly_sales.copy()
monthly_sales_display['month_str'] = monthly_sales_display['order_ym'].dt.
 ↪strftime('%Y-%m')

# To address outliers, clip growth rates to a reasonable range (e.g., -200% to␣
 ↪+200%)
clip_low, clip_high = -2, 2  # -200% to +200%
monthly_sales_display['pct_growth_clipped'] =␣
 ↪monthly_sales_display['pct_growth'].clip(clip_low, clip_high)

# Create Plotly boxplot with clipped data
fig_box = go.Figure()
```

```
months = monthly_sales_display['month_str'].unique()
months_sorted = sorted(months)

for m in months_sorted:
    data = monthly_sales_display.loc[monthly_sales_display['month_str'] == m,␣
 ↪'pct_growth_clipped']
    fig_box.add_trace(go.Box(
        y=data,
        name=m,
        boxpoints='outliers',
        marker_color='skyblue',
        line_color='blue',
        showlegend=False
    ))

fig_box.add_hline(y=0, line_dash="dash", line_color="red",
                  annotation_text="0% Growth Line", annotation_position="top␣
 ↪right")

fig_box.update_layout(
    title='Distribution of Monthly Growth Rates by Month (All Sellers, Clipped␣
 ↪to ±200%)',
    xaxis_title='Month',
    yaxis_title='Growth Rate (%)',
    height=600,
    margin=dict(t=80, b=80, l=80, r=40),
    xaxis_tickangle=45,
    yaxis=dict(
        tickformat=".0%",
        range=[clip_low, clip_high]
    )
)

fig_box.show()

print("Boxplot shows: median (line), Q25-Q75 (box), min-max (whiskers), and␣
 ↪outliers (dots)")
print("Y-axis clipped to ±200% to reduce the effect of extreme outliers.")
print()
```

```
=== MONTHLY GROWTH IN SALES BY SELLER ===

Using 98207 orders with valid statuses: ['delivered', 'shipped', 'invoiced',
'processing', 'created', 'approved']

Monthly sales data: 23460 seller-month combinations
```

```
Sample of monthly growth data:
                          seller_id   order_ym    sales  pct_growth
0  0015a82c2db000af6aaaf3ae2ecb0532 2017-06-01   895.00    0.000000
1  0015a82c2db000af6aaaf3ae2ecb0532 2017-10-01   895.00    0.000000
2  0015a82c2db000af6aaaf3ae2ecb0532 2018-07-01   895.00    0.000000
3  001cca7ae9ae17fb1caed9dfb1094831 2017-01-01   308.00    0.000000
4  001cca7ae9ae17fb1caed9dfb1094831 2017-02-01   228.00   -0.259740
5  001cca7ae9ae17fb1caed9dfb1094831 2017-03-01   267.90    0.175000
6  001cca7ae9ae17fb1caed9dfb1094831 2017-04-01   679.90    1.537887
7  001cca7ae9ae17fb1caed9dfb1094831 2017-05-01   977.98    0.438417
8  001cca7ae9ae17fb1caed9dfb1094831 2017-06-01   765.90   -0.216855
9  001cca7ae9ae17fb1caed9dfb1094831 2017-07-01  1124.90    0.468730

=== MONTHLY GROWTH SUMMARY ===

Growth rate statistics:
count    23460.000000
mean         0.743377
std          4.359232
min         -0.995653
25%         -0.325218
50%          0.000000
75%          0.648250
max        235.617309
Name: pct_growth, dtype: float64

Growth distribution:
  Positive growth: 9932 seller-months (42.3%)
  Negative growth: 9041 seller-months (38.5%)
  Zero growth: 4487 seller-months (19.1%)

Top 10 highest growth rates:
 1. ed4acab3… - 2018-06 - Growth: 23561.7% - Sales: $4,729.98
 2. 512d298a… - 2017-12 - Growth: 23179.7% - Sales: $4,399.87
 3. 582d4f86… - 2017-07 - Growth: 17390.7% - Sales: $1,047.69
 4. 3bfba5a7… - 2018-01 - Growth: 16821.8% - Sales: $1,319.90
 5. e59aa562… - 2018-01 - Growth: 14528.0% - Sales: $3,657.00
 6. 784ba75d… - 2017-06 - Growth: 12511.5% - Sales: $1,259.89
 7. acadd4d3… - 2017-11 - Growth: 10662.5% - Sales: $861.00
 8. 784ba75d… - 2018-02 - Growth: 9716.9% - Sales: $1,078.88
 9. 5670f4db… - 2018-06 - Growth: 9367.9% - Sales: $1,032.95
10. 05ff92fe… - 2017-11 - Growth: 8383.3% - Sales: $1,823.90

Top 10 lowest growth rates:
 1. 821fb029… - 2018-04 - Growth: -99.6% - Sales: $17.90
 2. 612a743d… - 2018-01 - Growth: -99.0% - Sales: $28.90
 3. ea566164… - 2018-08 - Growth: -98.8% - Sales: $41.90
 4. 05ff92fe… - 2017-09 - Growth: -98.8% - Sales: $21.50
```

```
 5. 784ba75d… - 2018-08 - Growth: -98.6% - Sales: $37.98
 6. 0307f756… - 2018-05 - Growth: -98.6% - Sales: $16.90
 7. 3bfba5a7… - 2017-12 - Growth: -98.5% - Sales: $7.80
 8. 0e44d110… - 2017-10 - Growth: -98.5% - Sales: $2.29
 9. cd06602b… - 2018-04 - Growth: -98.4% - Sales: $12.00
10. e59aa562… - 2017-12 - Growth: -98.4% - Sales: $25.00

=== MONTHLY GROWTH DISTRIBUTION - BOXPLOT BY MONTH (LOG SCALE/CLIPPED) ===


Boxplot shows: median (line), Q25-Q75 (box), min-max (whiskers), and outliers
(dots)
Y-axis clipped to ±200% to reduce the effect of extreme outliers.
```

1.13 9. Python Task 3 — Customer Cohort Analysis (first order month)

We build cohorts by **first purchase month** per `customer_unique_id`. Then, for each subsequent month, we measure **activity** (unique customers ordering). We compute **retention rate** = active customers in month k / cohort size. Heatmaps are displayed, as per the requirement. Below that is additional comprehensive analyses with a final executive summary.

[19]:
```
# Customer Cohort Analysis
print("=== CUSTOMER COHORT ANALYSIS ===")
print()

# Use filtered orders
valid_statuses = ['delivered', 'shipped', 'invoiced', 'processing', 'created',␣
 ↪'approved']
filtered_orders = orders[orders['order_status'].isin(valid_statuses)].copy()

# Create customer cohorts based on first purchase month
filtered_orders['order_purchase_timestamp'] = pd.
 ↪to_datetime(filtered_orders['order_purchase_timestamp'])
customer_first_purchase = (
    filtered_orders
    .groupby('customer_id', as_index=False)
    .agg(first_purchase_date=('order_purchase_timestamp', 'min'))
)
customer_first_purchase['cohort_month'] =␣
 ↪customer_first_purchase['first_purchase_date'].dt.to_period('M').dt.
 ↪to_timestamp()

print(f"Created cohorts for {len(customer_first_purchase)} unique customers")
print()

# Create order-level cohort data
order_cohort_data = (
```

```
    filtered_orders
    .merge(customer_first_purchase[['customer_id', 'cohort_month']],␣
 ↪on='customer_id', how='left')
    .assign(
        order_month=filtered_orders['order_purchase_timestamp'].dt.
 ↪to_period('M').dt.to_timestamp()
    )
    .merge(order_items, on='order_id', how='inner')
)

# Calculate period number (months since first purchase)
# Avoid IntCastingNaNError by dropping rows with missing cohort/order month␣
 ↪before astype(int)
period_number = (
    (order_cohort_data['order_month'] - order_cohort_data['cohort_month'])
    / pd.Timedelta(days=30.44)
).round()

order_cohort_data = order_cohort_data.assign(period_number=period_number)
order_cohort_data = order_cohort_data[order_cohort_data['period_number'].
 ↪notna()]
order_cohort_data['period_number'] = order_cohort_data['period_number'].
 ↪astype(int)

# Filter to reasonable periods
order_cohort_data = order_cohort_data[order_cohort_data['period_number'].
 ↪between(0, 24)]

print(f"Order-level cohort data: {len(order_cohort_data)} order-item␣
 ↪combinations")
print()

# Calculate cohort metrics
cohort_metrics = (
    order_cohort_data
    .groupby(['cohort_month', 'period_number'], as_index=False)
    .agg(
        unique_customers=('customer_id', 'nunique'),
        total_sales=('price', 'sum')
    )
    .sort_values(['cohort_month', 'period_number'])
)

# Get cohort sizes
cohort_sizes = (
    customer_first_purchase
```

```
    .groupby('cohort_month', as_index=False)
    .agg(cohort_size=('customer_id', 'nunique'))
    .sort_values('cohort_month')
)

# Calculate retention rates
retention_data = cohort_metrics.merge(cohort_sizes, on='cohort_month',␣
 ↪how='left')
retention_data['retention_rate'] = retention_data['unique_customers'] /␣
 ↪retention_data['cohort_size']

print("Sample retention data:")
print(retention_data.head(10))
print()
```

```
=== CUSTOMER COHORT ANALYSIS ===

Created cohorts for 98207 unique customers

Order-level cohort data: 58668 order-item combinations

Sample retention data:
  cohort_month  period_number  unique_customers  total_sales  cohort_size
retention_rate
0   2016-09-01             12                 1       134.97            2
0.500000
1   2016-09-01             21                 1        72.89            2
0.500000
2   2016-10-01              3                 1       149.90          293
0.003413
3   2016-10-01              4                 8      1525.57          293
0.027304
4   2016-10-01              5                10      1200.13          293
0.034130
5   2016-10-01              6                 7      1367.58          293
0.023891
6   2016-10-01              7                17      2632.56          293
0.058020
7   2016-10-01              8                 9      1632.88          293
0.030717
8   2016-10-01              9                10      1941.47          293
0.034130
9   2016-10-01             10                10      1862.17          293
0.034130
```

[20]:
```
# Create Cohort Heatmaps
print("=== COHORT HEATMAPS ===")
print()

# Create sales heatmap
sales_heatmap_data = retention_data.pivot(
    index='cohort_month',
    columns='period_number',
    values='total_sales'
).fillna(0)

sales_heatmap_data.index = sales_heatmap_data.index.strftime('%Y-%m')

fig_sales = px.imshow(
    sales_heatmap_data,
    title='Total Sales by Cohort and Period',
    labels=dict(x="Period (Months Since First Purchase)", y="Cohort Month",␣
 ↪color="Total sales ($)"),
    color_continuous_scale='Viridis',
    aspect='auto'
)

fig_sales.update_layout(
    height=600,
    margin=dict(t=80, b=80, l=120, r=40)
)

fig_sales.show()

# Create customer count heatmap
customer_heatmap_data = retention_data.pivot(
    index='cohort_month',
    columns='period_number',
    values='unique_customers'
).fillna(0)

customer_heatmap_data.index = customer_heatmap_data.index.strftime('%Y-%m')

fig_customers = px.imshow(
    customer_heatmap_data,
    title='Active Customers by Cohort and Period',
    labels=dict(x="Period (Months Since First Purchase)", y="Cohort Month",␣
 ↪color="Active Customers"),
    color_continuous_scale='Blues',
    aspect='auto'
)
```

```
fig_customers.update_layout(
    height=600,
    margin=dict(t=80, b=80, l=120, r=40)
)

fig_customers.show()
print()
```

```
=== COHORT HEATMAPS ===
```

[21]:
```
### Comprehensive Cohort Metrics and Business Insights

print("=== ADVANCED COHORT ANALYSIS ===")
print()

# 1. Cohort Size Analysis
print("1. COHORT SIZE ANALYSIS")
print("=" * 50)
cohort_size_analysis = (
    retention_data
    .groupby('cohort_month', as_index=False)
    .agg(
        cohort_size=('cohort_size', 'first'),
        avg_retention_3m=('retention_rate', lambda x: x[retention_data.loc[x.
 ↪index, 'period_number'] == 3].mean()),
        avg_retention_6m=('retention_rate', lambda x: x[retention_data.loc[x.
 ↪index, 'period_number'] == 6].mean()),
        avg_retention_12m=('retention_rate', lambda x: x[retention_data.loc[x.
 ↪index, 'period_number'] == 12].mean())
    )
    .sort_values('cohort_month')
)

cohort_size_analysis['cohort_month_str'] = cohort_size_analysis['cohort_month'].
 ↪dt.strftime('%Y-%m')
print("Cohort sizes and key retention metrics:")
print(cohort_size_analysis[['cohort_month_str', 'cohort_size',␣
 ↪'avg_retention_3m', 'avg_retention_6m', 'avg_retention_12m']].round(3))
print()

# Visualize cohort sizes over time
fig_cohort_sizes = px.bar(
    cohort_size_analysis,
    x='cohort_month_str',
    y='cohort_size',
```

```
    title='Customer Cohort Sizes Over Time',
    labels={'cohort_month_str': 'Cohort Month', 'cohort_size': 'Number of␣
 ↪Customers'}
)
fig_cohort_sizes.update_layout(xaxis_tickangle=45, height=500)
fig_cohort_sizes.show()

# 2. Retention Rate Analysis by Period
print("2. RETENTION RATE ANALYSIS BY PERIOD")
print("=" * 50)
retention_by_period = (
    retention_data
    .groupby('period_number', as_index=False)
    .agg(
        avg_retention=('retention_rate', 'mean'),
        median_retention=('retention_rate', 'median'),
        min_retention=('retention_rate', 'min'),
        max_retention=('retention_rate', 'max'),
        std_retention=('retention_rate', 'std')
    )
    .round(3)
)

print("Retention rate statistics by period:")
print(retention_by_period.head(15))
print()

# Visualize retention decay
fig_retention_decay = px.line(
    retention_by_period,
    x='period_number',
    y='avg_retention',
    title='Average Retention Rate Decay Over Time',
    labels={'period_number': 'Months Since First Purchase', 'avg_retention':␣
 ↪'Average Retention Rate'},
    markers=True
)
fig_retention_decay.add_hline(y=0.1, line_dash="dash", line_color="red",
                              annotation_text="10% Retention Threshold")
fig_retention_decay.update_layout(height=500)
fig_retention_decay.show()

# 3. Revenue Analysis by Cohort
print("3. REVENUE ANALYSIS BY COHORT")
print("=" * 50)
revenue_analysis = (
    retention_data
```

```
    .groupby(['cohort_month', 'period_number'], as_index=False)
    .agg(
        total_revenue=('total_sales', 'sum'),
        avg_revenue_per_customer=('total_sales', 'sum'),
        active_customers=('unique_customers', 'sum')
    )
)

# Calculate cumulative revenue by cohort
revenue_analysis['cumulative_revenue'] = (
    revenue_analysis
    .groupby('cohort_month')['total_revenue']
    .cumsum()
)

# Calculate revenue per active customer
revenue_analysis['revenue_per_active_customer'] = (
    revenue_analysis['total_revenue'] / revenue_analysis['active_customers'].
 ↪replace(0, 1)
)

print("Sample revenue analysis:")
print(revenue_analysis.head(10))
print()

# 4. Cohort Performance Comparison
print("4. COHORT PERFORMANCE COMPARISON")
print("=" * 50)

# Calculate key metrics for each cohort
cohort_performance = (
    retention_data
    .groupby('cohort_month', as_index=False)
    .agg(
        cohort_size=('cohort_size', 'first'),
        total_revenue=('total_sales', 'sum'),
        avg_retention_6m=('retention_rate', lambda x: x[retention_data.loc[x.
 ↪index, 'period_number'] == 6].mean() if len(x[retention_data.loc[x.index,␣
 ↪'period_number'] == 6]) > 0 else 0),
        avg_retention_12m=('retention_rate', lambda x: x[retention_data.loc[x.
 ↪index, 'period_number'] == 12].mean() if len(x[retention_data.loc[x.index,␣
 ↪'period_number'] == 12]) > 0 else 0),
        lifetime_value=('total_sales', 'sum')
    )
)
```

```
cohort_performance['revenue_per_customer'] =␣
 ↪cohort_performance['total_revenue'] / cohort_performance['cohort_size']
cohort_performance['cohort_month_str'] = cohort_performance['cohort_month'].dt.
 ↪strftime('%Y-%m')

# Rank cohorts by performance
cohort_performance['revenue_rank'] = cohort_performance['total_revenue'].
 ↪rank(ascending=False)
cohort_performance['retention_rank'] = cohort_performance['avg_retention_6m'].
 ↪rank(ascending=False)

print("Top 10 cohorts by total revenue:")
top_revenue_cohorts = cohort_performance.nlargest(10,␣
 ↪'total_revenue')[['cohort_month_str', 'cohort_size', 'total_revenue',␣
 ↪'revenue_per_customer', 'avg_retention_6m']]
print(top_revenue_cohorts.round(2))
print()

print("Top 10 cohorts by 6-month retention:")
top_retention_cohorts = cohort_performance.nlargest(10,␣
 ↪'avg_retention_6m')[['cohort_month_str', 'cohort_size', 'avg_retention_6m',␣
 ↪'total_revenue', 'revenue_per_customer']]
print(top_retention_cohorts.round(3))
print()

# 5. Statistical Analysis and Trends
print("5. STATISTICAL ANALYSIS AND TRENDS")
print("=" * 50)

# Calculate correlation between cohort size and retention
correlation_size_retention = cohort_performance['cohort_size'].
 ↪corr(cohort_performance['avg_retention_6m'])
print(f"Correlation between cohort size and 6-month retention:␣
 ↪{correlation_size_retention:.3f}")

# Calculate correlation between cohort size and revenue per customer
correlation_size_revenue = cohort_performance['cohort_size'].
 ↪corr(cohort_performance['revenue_per_customer'])
print(f"Correlation between cohort size and revenue per customer:␣
 ↪{correlation_size_revenue:.3f}")

# Identify trends
cohort_performance['year'] = cohort_performance['cohort_month'].dt.year
cohort_performance['month'] = cohort_performance['cohort_month'].dt.month

yearly_trends = (
```

```
    cohort_performance
    .groupby('year', as_index=False)
    .agg(
        avg_cohort_size=('cohort_size', 'mean'),
        avg_retention_6m=('avg_retention_6m', 'mean'),
        avg_revenue_per_customer=('revenue_per_customer', 'mean'),
        total_cohorts=('cohort_month', 'count')
    )
    .round(3)
)

print("\nYearly trends:")
print(yearly_trends)
print()

# 6. Business Insights and Recommendations
print("6. BUSINESS INSIGHTS AND RECOMMENDATIONS")
print("=" * 50)

# Calculate key business metrics
total_customers = cohort_performance['cohort_size'].sum()
total_revenue = cohort_performance['total_revenue'].sum()
avg_retention_6m = cohort_performance['avg_retention_6m'].mean()
avg_revenue_per_customer = cohort_performance['revenue_per_customer'].mean()

print(f"KEY BUSINESS METRICS:")
print(f"• Total customers acquired: {total_customers:,}")
print(f"• Total revenue generated: ${total_revenue:,.2f}")
print(f"• Average 6-month retention rate: {avg_retention_6m:.1%}")
print(f"• Average revenue per customer: ${avg_revenue_per_customer:.2f}")
print()

# Identify best and worst performing cohorts
best_cohort = cohort_performance.loc[cohort_performance['revenue_rank'].
 ↪idxmin()]
worst_cohort = cohort_performance.loc[cohort_performance['revenue_rank'].
 ↪idxmax()]

print(f"BEST PERFORMING COHORT: {best_cohort['cohort_month_str']}")
print(f"• Cohort size: {best_cohort['cohort_size']:,}")
print(f"• Total revenue: ${best_cohort['total_revenue']:,.2f}")
print(f"• Revenue per customer: ${best_cohort['revenue_per_customer']:.2f}")
print(f"• 6-month retention: {best_cohort['avg_retention_6m']:.1%}")
print()

print(f"WORST PERFORMING COHORT: {worst_cohort['cohort_month_str']}")
print(f"• Cohort size: {worst_cohort['cohort_size']:,}")
```

```
print(f"• Total revenue: ${worst_cohort['total_revenue']:,.2f}")
print(f"• Revenue per customer: ${worst_cohort['revenue_per_customer']:.2f}")
print(f"• 6-month retention: {worst_cohort['avg_retention_6m']:.1%}")
print()

# Calculate retention decay rate
retention_decay = retention_by_period[retention_by_period['period_number'].
  ↪between(3, 12)]['avg_retention'].pct_change().mean()
print(f"Average monthly retention decay rate (months 3-12): {retention_decay:.
  ↪1%}")
print()

print("ACTIONABLE RECOMMENDATIONS:")
print("1. Focus on improving 6-month retention rates")
print("2. Analyze successful cohorts to identify acquisition strategies")
print("3. Implement retention campaigns for customers in months 3-6")
print("4. Monitor cohort performance trends monthly")
print("5. Consider cohort-specific marketing strategies")
print()
```

```
=== ADVANCED COHORT ANALYSIS ===

1. COHORT SIZE ANALYSIS
==================================================
Cohort sizes and key retention metrics:
   cohort_month_str  cohort_size  avg_retention_3m  avg_retention_6m
avg_retention_12m
0           2016-09           2               NaN               NaN
0.500
1           2016-10         293             0.003             0.024
0.065
2           2016-12           1               NaN               NaN
NaN
3           2017-01         787             0.024             0.034
0.076
4           2017-02        1718             0.033             0.048
0.063
5           2017-03        2617             0.033             0.043
0.074
6           2017-04        2377             0.037             0.049
0.071
7           2017-05        3640             0.040             0.073
0.072
8           2017-06        3205             0.039             0.053
0.065
9           2017-07        3946             0.043             0.065
0.062
```

```
10          2017-08         4272            0.074           0.063
0.063
11          2017-09         4227            0.057           0.071
NaN
12          2017-10         4547            0.071           0.065
NaN
13          2017-11         7423            0.067           0.067
NaN
14          2017-12         5620            0.072           0.059
NaN
15          2018-01         7187            0.068           0.062
NaN
16          2018-02         6625            0.067           0.063
NaN
17          2018-03         7168            0.057             NaN
NaN
18          2018-04         6919            0.057             NaN
NaN
19          2018-05         6833            0.062             NaN
NaN
20          2018-06         6145              NaN             NaN
NaN
21          2018-07         6233              NaN             NaN
NaN
22          2018-08         6421              NaN             NaN
NaN


2. RETENTION RATE ANALYSIS BY PERIOD
================================================
Retention rate statistics by period:
    period_number  avg_retention  median_retention  min_retention  max_retention
std_retention
0               0          0.051             0.053          0.011          0.080
0.021
1               1          0.051             0.057          0.014          0.075
0.018
2               2          0.053             0.059          0.017          0.074
0.018
3               3          0.050             0.057          0.003          0.074
0.020
4               4          0.054             0.062          0.027          0.076
0.017
5               5          0.054             0.061          0.000          0.080
0.020
6               6          0.056             0.062          0.024          0.073
0.014
7               7          0.061             0.063          0.037          0.075
```

```
0.012
8               8          0.061             0.066          0.031          0.077
0.014
9               9          0.065             0.067          0.034          0.079
0.011
10             10          0.065             0.067          0.034          0.081
0.013
11             11          0.063             0.066          0.044          0.076
0.010
12             12          0.111             0.068          0.062          0.500
0.137
13             13          0.068             0.069          0.055          0.076
0.007
14             14          0.065             0.064          0.061          0.071
0.003


3. REVENUE ANALYSIS BY COHORT
==================================================
Sample revenue analysis:
  cohort_month  period_number  total_revenue  avg_revenue_per_customer
active_customers  cumulative_revenue  \
0   2016-09-01             12         134.97                    134.97
1              134.97
1   2016-09-01             21          72.89                     72.89
1              207.86
2   2016-10-01              3         149.90                    149.90
1              149.90
3   2016-10-01              4        1525.57                   1525.57
8             1675.47
4   2016-10-01              5        1200.13                   1200.13
10             2875.60
5   2016-10-01              6        1367.58                   1367.58
7             4243.18
6   2016-10-01              7        2632.56                   2632.56
17             6875.74
7   2016-10-01              8        1632.88                   1632.88
9             8508.62
8   2016-10-01              9        1941.47                   1941.47
10            10450.09
9   2016-10-01             10        1862.17                   1862.17
10            12312.26

   revenue_per_active_customer
0                   134.970000
1                    72.890000
2                   149.900000
3                   190.696250
```

```
4                   120.013000
5                   195.368571
6                   154.856471
7                   181.431111
8                   194.147000
9                   186.217000

4. COHORT PERFORMANCE COMPARISON
==================================================
Top 10 cohorts by total revenue:
   cohort_month_str  cohort_size  total_revenue  revenue_per_customer
avg_retention_6m
13          2017-11         7423      668758.58                 90.09
0.07
15          2018-01         7187      507322.04                 70.59
0.06
11          2017-09         4227      480476.72                113.67
0.07
12          2017-10         4547      464730.88                102.21
0.07
7           2017-05         3640      461771.32                126.86
0.07
10          2017-08         4272      453439.34                106.14
0.06
14          2017-12         5620      451077.25                 80.26
0.06
9           2017-07         3946      416065.72                105.44
0.07
16          2018-02         6625      390624.08                 58.96
0.06
17          2018-03         7168      379332.04                 52.92
0.00

Top 10 cohorts by 6-month retention:
   cohort_month_str  cohort_size  avg_retention_6m  total_revenue
revenue_per_customer
7           2017-05         3640             0.073      461771.32
126.860
11          2017-09         4227             0.071      480476.72
113.668
13          2017-11         7423             0.067      668758.58
90.093
9           2017-07         3946             0.065      416065.72
105.440
12          2017-10         4547             0.065      464730.88
102.206
16          2018-02         6625             0.063      390624.08
58.962
```

```
10          2017-08         4272            0.063      453439.34
106.142
15          2018-01         7187            0.062      507322.04
70.589
14          2017-12         5620            0.059      451077.25
80.263
8           2017-06         3205            0.053      365432.92
114.020

5. STATISTICAL ANALYSIS AND TRENDS
==================================================
Correlation between cohort size and 6-month retention: 0.007
Correlation between cohort size and revenue per customer: -0.571

Yearly trends:
   year  avg_cohort_size  avg_retention_6m  avg_revenue_per_customer
total_cohorts
0  2016           98.667             0.008                    88.589
3
1  2017         3698.250             0.057                   117.088
12
2  2018         6691.375             0.016                    40.230
8

6. BUSINESS INSIGHTS AND RECOMMENDATIONS
==================================================
KEY BUSINESS METRICS:
• Total customers acquired: 98,206
• Total revenue generated: $7,052,360.24
• Average 6-month retention rate: 3.6%
• Average revenue per customer: $86.64

BEST PERFORMING COHORT: 2017-11
• Cohort size: 7,423
• Total revenue: $668,758.58
• Revenue per customer: $90.09
• 6-month retention: 6.7%

WORST PERFORMING COHORT: 2016-12
• Cohort size: 1
• Total revenue: $10.90
• Revenue per customer: $10.90
• 6-month retention: 0.0%

Average monthly retention decay rate (months 3-12): 11.1%

ACTIONABLE RECOMMENDATIONS:
1. Focus on improving 6-month retention rates
```

2. Analyze successful cohorts to identify acquisition strategies
3. Implement retention campaigns for customers in months 3-6
4. Monitor cohort performance trends monthly
5. Consider cohort-specific marketing strategies

[22]:
```
## Advanced Cohort Analytics - Statistical Modeling and Predictive Insights

print("=== ADVANCED COHORT ANALYTICS ===")
print()

# 1. Cohort Lifecycle Analysis
print("1. COHORT LIFECYCLE ANALYSIS")
print("=" * 50)

# Calculate cohort lifecycle metrics
cohort_lifecycle = (
    retention_data
    .groupby('cohort_month', as_index=False)
    .agg(
        cohort_size=('cohort_size', 'first'),
        first_period_revenue=('total_sales', lambda x: x[retention_data.loc[x.
 ↪index, 'period_number'] == 0].sum()),
        peak_revenue_period=('total_sales', 'idxmax'),
        total_lifetime_revenue=('total_sales', 'sum'),
        avg_revenue_per_period=('total_sales', 'mean'),
        revenue_volatility=('total_sales', 'std')
    )
)

# Calculate peak revenue period number
cohort_lifecycle['peak_period'] = retention_data.
 ↪loc[cohort_lifecycle['peak_revenue_period'], 'period_number'].values
cohort_lifecycle['cohort_month_str'] = cohort_lifecycle['cohort_month'].dt.
 ↪strftime('%Y-%m')

print("Cohort lifecycle analysis:")
print(cohort_lifecycle[['cohort_month_str', 'cohort_size',␣
 ↪'first_period_revenue', 'peak_period', 'total_lifetime_revenue',␣
 ↪'avg_revenue_per_period']].round(2))
print()

# 2. Cohort Segmentation Analysis
print("2. COHORT SEGMENTATION ANALYSIS")
print("=" * 50)

# Segment cohorts by performance
```

```
# Use include_lowest=True and duplicates='drop' to avoid ValueError if there␣
 ↪are duplicate bin edges
cohort_performance['performance_segment'] = pd.cut(
    cohort_performance['revenue_per_customer'],
    bins=3,
    labels=['Low Value', 'Medium Value', 'High Value'],
    include_lowest=True,
    duplicates='drop'
)

# Segment by retention
cohort_performance['retention_segment'] = pd.cut(
    cohort_performance['avg_retention_6m'],
    bins=3,
    labels=['Low Retention', 'Medium Retention', 'High Retention'],
    include_lowest=True,
    duplicates='drop'
)

# Remove rows with NA in either segment (can happen if all values are the same␣
 ↪or bins collapse)
cohort_segmentation = cohort_performance.dropna(subset=['performance_segment',␣
 ↪'retention_segment'])

# Create cohort matrix
cohort_matrix = (
    cohort_segmentation
    .groupby(['performance_segment', 'retention_segment'], as_index=False,␣
 ↪observed=True)
    .agg(
        count=('cohort_month', 'count'),
        avg_cohort_size=('cohort_size', 'mean'),
        avg_revenue_per_customer=('revenue_per_customer', 'mean'),
        avg_retention_6m=('avg_retention_6m', 'mean')
    )
    .round(2)
)

print("Cohort segmentation matrix:")
print(cohort_matrix)
print()

# 3. Time Series Analysis of Cohort Performance
print("3. TIME SERIES ANALYSIS OF COHORT PERFORMANCE")
print("=" * 50)

# Calculate monthly cohort performance trends
```

```
monthly_cohort_trends = (
    cohort_performance
    .groupby(['year', 'month'], as_index=False)
    .agg(
        avg_cohort_size=('cohort_size', 'mean'),
        avg_revenue_per_customer=('revenue_per_customer', 'mean'),
        avg_retention_6m=('avg_retention_6m', 'mean'),
        total_cohorts=('cohort_month', 'count')
    )
    .sort_values(['year', 'month'])
)

monthly_cohort_trends['period'] = monthly_cohort_trends['year'].astype(str) +␣
 ↪'-' + monthly_cohort_trends['month'].astype(str).str.zfill(2)

print("Monthly cohort performance trends:")
print(monthly_cohort_trends[['period', 'avg_cohort_size',␣
 ↪'avg_revenue_per_customer', 'avg_retention_6m', 'total_cohorts']].round(3))
print()

# Visualize trends
fig_trends = px.line(
    monthly_cohort_trends,
    x='period',
    y=['avg_cohort_size', 'avg_revenue_per_customer', 'avg_retention_6m'],
    title='Monthly Cohort Performance Trends',
    labels={'value': 'Metric Value', 'variable': 'Metric'}
)
fig_trends.update_layout(xaxis_tickangle=45, height=600)
fig_trends.show()

# 4. Cohort Comparison and Benchmarking
print("4. COHORT COMPARISON AND BENCHMARKING")
print("=" * 50)

# Calculate benchmarks
benchmarks = {
    'median_cohort_size': cohort_performance['cohort_size'].median(),
    'median_revenue_per_customer': cohort_performance['revenue_per_customer'].
 ↪median(),
    'median_retention_6m': cohort_performance['avg_retention_6m'].median(),
    'top_quartile_revenue': cohort_performance['revenue_per_customer'].
 ↪quantile(0.75),
    'top_quartile_retention': cohort_performance['avg_retention_6m'].quantile(0.
 ↪75)
}
```

```
print("Benchmark metrics:")
for metric, value in benchmarks.items():
    print(f"• {metric}: {value:.2f}")
print()

# Identify cohorts above benchmarks
above_benchmark = cohort_performance[
    (cohort_performance['revenue_per_customer'] >␣
 ↪benchmarks['top_quartile_revenue']) &
    (cohort_performance['avg_retention_6m'] >␣
 ↪benchmarks['top_quartile_retention'])
]

print(f"Cohorts above both revenue and retention benchmarks:␣
 ↪{len(above_benchmark)}")
if len(above_benchmark) > 0:
    print("High-performing cohorts:")
    print(above_benchmark[['cohort_month_str', 'cohort_size',␣
 ↪'revenue_per_customer', 'avg_retention_6m']].round(2))
print()

# 5. Predictive Insights and Forecasting
print("5. PREDICTIVE INSIGHTS AND FORECASTING")
print("=" * 50)

# Calculate cohort growth rates
cohort_performance['cohort_size_growth'] = cohort_performance['cohort_size'].
 ↪pct_change()
cohort_performance['revenue_growth'] =␣
 ↪cohort_performance['revenue_per_customer'].pct_change()

# Identify growth trends
recent_cohorts = cohort_performance[cohort_performance['cohort_month'] >=␣
 ↪'2018-01-01']
if len(recent_cohorts) > 1:
    avg_growth_rate = recent_cohorts['cohort_size_growth'].mean()
    avg_revenue_growth = recent_cohorts['revenue_growth'].mean()

    print(f"Average cohort size growth rate (2018+): {avg_growth_rate:.1%}")
    print(f"Average revenue per customer growth rate (2018+):␣
 ↪{avg_revenue_growth:.1%}")

    # Simple trend projection
    if avg_growth_rate > 0:
        print("  Positive cohort size growth trend detected")
    else:
```

```
        print("  Negative cohort size growth trend detected")

    if avg_revenue_growth > 0:
        print("  Positive revenue per customer growth trend detected")
    else:
        print("  Negative revenue per customer growth trend detected")
print()

# 6. Cohort Health Score
print("6. COHORT HEALTH SCORE")
print("=" * 50)

# Calculate composite health score for each cohort
cohort_performance['health_score'] = (
    (cohort_performance['revenue_per_customer'] /␣
 ↪cohort_performance['revenue_per_customer'].max()) * 0.4 +
    (cohort_performance['avg_retention_6m'] /␣
 ↪cohort_performance['avg_retention_6m'].max()) * 0.4 +
    (cohort_performance['cohort_size'] / cohort_performance['cohort_size'].
 ↪max()) * 0.2
)

cohort_performance['health_grade'] = pd.cut(
    cohort_performance['health_score'],
    bins=[0, 0.3, 0.6, 0.8, 1.0],
    labels=['D', 'C', 'B', 'A'],
    include_lowest=True,
    duplicates='drop'
)

# Show health score distribution
health_distribution = cohort_performance['health_grade'].value_counts().
 ↪sort_index()
print("Cohort health grade distribution:")
print(health_distribution)
print()

# Show top and bottom cohorts by health score
print("Top 5 cohorts by health score:")
top_health = cohort_performance.nlargest(5,␣
 ↪'health_score')[['cohort_month_str', 'health_score', 'health_grade',␣
 ↪'revenue_per_customer', 'avg_retention_6m']]
print(top_health.round(3))
print()

print("Bottom 5 cohorts by health score:")
```

```
bottom_health = cohort_performance.nsmallest(5,␣
 ↪'health_score')[['cohort_month_str', 'health_score', 'health_grade',␣
 ↪'revenue_per_customer', 'avg_retention_6m']]
print(bottom_health.round(3))
print()

# 7. Cohort Health Score Analysis
print("7. COHORT HEALTH SCORE ANALYSIS")
print("=" * 50)

# Calculate composite health score for each cohort
cohort_performance['health_score'] = (
    (cohort_performance['revenue_per_customer'] /␣
 ↪cohort_performance['revenue_per_customer'].max()) * 0.4 +
    (cohort_performance['avg_retention_6m'] /␣
 ↪cohort_performance['avg_retention_6m'].max()) * 0.4 +
    (cohort_performance['cohort_size'] / cohort_performance['cohort_size'].
 ↪max()) * 0.2
)

cohort_performance['health_grade'] = pd.cut(
    cohort_performance['health_score'],
    bins=[0, 0.3, 0.6, 0.8, 1.0],
    labels=['D', 'C', 'B', 'A']
)

# Show health score distribution
health_distribution = cohort_performance['health_grade'].value_counts().
 ↪sort_index()
print("Cohort health grade distribution:")
print(health_distribution)
print()

# Show top and bottom cohorts by health score
print("Top 5 cohorts by health score:")
top_health = cohort_performance.nlargest(5,␣
 ↪'health_score')[['cohort_month_str', 'health_score', 'health_grade',␣
 ↪'revenue_per_customer', 'avg_retention_6m']]
print(top_health.round(3))
print()

print("Bottom 5 cohorts by health score:")
bottom_health = cohort_performance.nsmallest(5,␣
 ↪'health_score')[['cohort_month_str', 'health_score', 'health_grade',␣
 ↪'revenue_per_customer', 'avg_retention_6m']]
print(bottom_health.round(3))
```

```
print()
```

```
=== ADVANCED COHORT ANALYTICS ===

1. COHORT LIFECYCLE ANALYSIS
==================================================
Cohort lifecycle analysis:
   cohort_month_str  cohort_size  first_period_revenue  peak_period
total_lifetime_revenue  avg_revenue_per_period
0           2016-09           2                  0.00           12
207.86                  103.93
1           2016-10         293                  0.00           20
44224.46                2211.22
2           2016-12           1                  0.00           19
10.90                   10.90
3           2017-01         787               2678.31           13
119236.61                5961.83
4           2017-02        1718               3268.78           18
240066.74               12635.09
5           2017-03        2617              10011.91            8
354554.49               19697.47
6           2017-04        2377               8351.98            9
331915.22               19524.42
7           2017-05        3640              18346.20           11
461771.32               28860.71
8           2017-06        3205              14720.71            7
365432.92               24362.19
9           2017-07        3946              25619.70            4
416065.72               29718.98
10          2017-08        4272              21310.47            5
453439.34               34879.95
11          2017-09        4227              21307.29           11
480476.72               40039.73
12          2017-10        4547              31477.21            1
464730.88               42248.26
13          2017-11        7423              81588.54            0
668758.58               66875.86
14          2017-12        5620              52443.23            8
451077.25               50119.69
15          2018-01        7187              69347.65            2
507322.04               63415.26
16          2018-02        6625              58992.06            1
390624.08               55803.44
17          2018-03        7168              67541.76            2
379332.04               63222.01
18          2018-04        6919              69865.36            4
327124.42               54520.74
19          2018-05        6833              68274.97            2
```

```
261117.71                65279.43
20          2018-06         6145              56435.65            2
167637.95                55879.32
21          2018-07         6233              55210.59            1
116749.20                58374.60
22          2018-08         6421              50483.79            0
50483.79                50483.79

2. COHORT SEGMENTATION ANALYSIS
==================================================
Cohort segmentation matrix:
  performance_segment retention_segment  count  avg_cohort_size
avg_revenue_per_customer  avg_retention_6m
0           Low Value     Low Retention      7          5674.29
29.03              0.00
1        Medium Value    High Retention      5          6280.40
80.42              0.06
2          High Value     Low Retention      2           147.50
127.43             0.01
3          High Value  Medium Retention      3          1707.33
142.24             0.04
4          High Value    High Retention      6          3611.17
117.63             0.06

3. TIME SERIES ANALYSIS OF COHORT PERFORMANCE
==================================================
Monthly cohort performance trends:
     period  avg_cohort_size  avg_revenue_per_customer  avg_retention_6m
total_cohorts
0   2016-09              2.0                   103.930             0.000
1
1   2016-10            293.0                   150.937             0.024
1
2   2016-12              1.0                    10.900             0.000
1
3   2017-01            787.0                   151.508             0.034
1
4   2017-02           1718.0                   139.736             0.048
1
5   2017-03           2617.0                   135.481             0.043
1
6   2017-04           2377.0                   139.636             0.049
1
7   2017-05           3640.0                   126.860             0.073
1
8   2017-06           3205.0                   114.020             0.053
1
9   2017-07           3946.0                   105.440             0.065
```

```
1
10  2017-08           4272.0                    106.142             0.063
1
11  2017-09           4227.0                    113.668             0.071
1
12  2017-10           4547.0                    102.206             0.065
1
13  2017-11           7423.0                     90.093             0.067
1
14  2017-12           5620.0                     80.263             0.059
1
15  2018-01           7187.0                     70.589             0.062
1
16  2018-02           6625.0                     58.962             0.063
1
17  2018-03           7168.0                     52.920             0.000
1
18  2018-04           6919.0                     47.279             0.000
1
19  2018-05           6833.0                     38.214             0.000
1
20  2018-06           6145.0                     27.280             0.000
1
21  2018-07           6233.0                     18.731             0.000
1
22  2018-08           6421.0                      7.862             0.000
1


4. COHORT COMPARISON AND BENCHMARKING
==================================================
Benchmark metrics:
• median_cohort_size: 4272.00
• median_revenue_per_customer: 102.21
• median_retention_6m: 0.05
• top_quartile_revenue: 120.44
• top_quartile_retention: 0.06

Cohorts above both revenue and retention benchmarks: 1
High-performing cohorts:
  cohort_month_str  cohort_size  revenue_per_customer  avg_retention_6m
7          2017-05         3640                126.86              0.07

5. PREDICTIVE INSIGHTS AND FORECASTING
==================================================
Average cohort size growth rate (2018+): 2.2%
Average revenue per customer growth rate (2018+): -23.3%
  Positive cohort size growth trend detected
```

```
  Negative revenue per customer growth trend detected

6. COHORT HEALTH SCORE
==================================================
Cohort health grade distribution:
health_grade
D     6
C     3
B    11
A     3
Name: count, dtype: int64

Top 5 cohorts by health score:
   cohort_month_str  health_score health_grade  revenue_per_customer
avg_retention_6m
7           2017-05         0.833            A               126.860
0.073
13          2017-11         0.805            A                90.093
0.067
11          2017-09         0.804            A               113.668
0.071
12          2017-10         0.750            B               102.206
0.065
9           2017-07         0.743            B               105.440
0.065

Bottom 5 cohorts by health score:
   cohort_month_str  health_score health_grade  revenue_per_customer
avg_retention_6m
2           2016-12         0.029            D                10.900
0.0
22          2018-08         0.194            D                 7.862
0.0
21          2018-07         0.217            D                18.731
0.0
20          2018-06         0.238            D                27.280
0.0
0           2016-09         0.274            D               103.930
0.0

7. COHORT HEALTH SCORE ANALYSIS
==================================================
Cohort health grade distribution:
health_grade
D     6
C     3
B    11
A     3
```

```
Name: count, dtype: int64

Top 5 cohorts by health score:
   cohort_month_str  health_score health_grade  revenue_per_customer
avg_retention_6m
7           2017-05         0.833            A                126.860
0.073
13          2017-11         0.805            A                 90.093
0.067
11          2017-09         0.804            A                113.668
0.071
12          2017-10         0.750            B                102.206
0.065
9           2017-07         0.743            B                105.440
0.065

Bottom 5 cohorts by health score:
   cohort_month_str  health_score health_grade  revenue_per_customer
avg_retention_6m
2           2016-12         0.029            D                 10.900
0.0
22          2018-08         0.194            D                  7.862
0.0
21          2018-07         0.217            D                 18.731
0.0
20          2018-06         0.238            D                 27.280
0.0
0           2016-09         0.274            D                103.930
0.0
```

[23]:
```
## 9.3. Advanced Cohort Visualizations and Final Summary

print("=== ADVANCED COHORT VISUALIZATIONS ===")
print()

# 1. Cohort Performance Dashboard
print("1. COHORT PERFORMANCE DASHBOARD")
print("=" * 50)

# Create a comprehensive cohort performance visualization
fig_dashboard = go.Figure()

# Add cohort size as bars
fig_dashboard.add_trace(go.Bar(
    x=cohort_performance['cohort_month_str'],
    y=cohort_performance['cohort_size'],
```

```
    name='Cohort Size',
    yaxis='y',
    marker_color='lightblue'
))

# Add revenue per customer as line
fig_dashboard.add_trace(go.Scatter(
    x=cohort_performance['cohort_month_str'],
    y=cohort_performance['revenue_per_customer'],
    name='Revenue per Customer',
    yaxis='y2',
    line=dict(color='red', width=2),
    mode='lines+markers'
))

# Add retention rate as line
fig_dashboard.add_trace(go.Scatter(
    x=cohort_performance['cohort_month_str'],
    y=cohort_performance['avg_retention_6m'],
    name='6-Month Retention',
    yaxis='y3',
    line=dict(color='green', width=2),
    mode='lines+markers'
))

# Update layout
fig_dashboard.update_layout(
    title='Cohort Performance Dashboard',
    xaxis=dict(title='Cohort Month'),
    yaxis=dict(title='Cohort Size', side='left'),
    yaxis2=dict(title='Revenue per Customer ($)', side='right', overlaying='y'),
    yaxis3=dict(title='6-Month Retention Rate', side='right', overlaying='y',␣
 ↪position=0.85),
    height=600,
    margin=dict(t=80, b=80, l=80, r=120)
)

fig_dashboard.show()

# 2. Cohort Health Score Heatmap
print("2. COHORT HEALTH SCORE HEATMAP")
print("=" * 50)

# Create health score heatmap (health_score should already be calculated in␣
 ↪previous cell)
if 'health_score' in cohort_performance.columns:
    health_heatmap_data = cohort_performance.pivot_table(
```

```
        index=cohort_performance['cohort_month'].dt.year,
        columns=cohort_performance['cohort_month'].dt.month,
        values='health_score',
        aggfunc='mean'
    ).fillna(0)

    fig_health_heatmap = px.imshow(
        health_heatmap_data,
        title='Cohort Health Score by Year and Month',
        labels=dict(x="Month", y="Year", color="Health Score"),
        color_continuous_scale='RdYlGn',
        aspect='auto'
    )

    fig_health_heatmap.update_layout(
        height=500,
        margin=dict(t=80, b=80, l=80, r=40)
    )

    fig_health_heatmap.show()
else:
    print("Health score not calculated yet. Please run previous cells first.")

# 3. Cohort Retention Decay Analysis
print("3. COHORT RETENTION DECAY ANALYSIS")
print("=" * 50)

# Calculate retention decay for each cohort
retention_decay_analysis = []

for cohort in retention_data['cohort_month'].unique():
    cohort_data = retention_data[retention_data['cohort_month'] == cohort].
 ↪sort_values('period_number')

    # Calculate decay rate (slope of retention curve)
    if len(cohort_data) > 3:
        periods = cohort_data['period_number'].values
        retention_rates = cohort_data['retention_rate'].values

        # Calculate slope using linear regression
        slope = np.polyfit(periods, retention_rates, 1)[0]

        retention_decay_analysis.append({
            'cohort_month': cohort,
            'cohort_month_str': cohort.strftime('%Y-%m'),
            'decay_rate': slope,
```

```
            'initial_retention': retention_rates[0] if len(retention_rates) > 0␣
 ↪else 0,
            'final_retention': retention_rates[-1] if len(retention_rates) > 0␣
 ↪else 0
        })

retention_decay_df = pd.DataFrame(retention_decay_analysis)

if len(retention_decay_df) > 0:
    print("Cohort retention decay analysis:")
    print(retention_decay_df[['cohort_month_str', 'decay_rate',␣
 ↪'initial_retention', 'final_retention']].round(3))
    print()

    # Visualize decay rates
    fig_decay = px.bar(
        retention_decay_df,
        x='cohort_month_str',
        y='decay_rate',
        title='Cohort Retention Decay Rates',
        labels={'cohort_month_str': 'Cohort Month', 'decay_rate': 'Decay Rate␣
 ↪(slope)'},
        color='decay_rate',
        color_continuous_scale='Reds'
    )
    fig_decay.update_layout(xaxis_tickangle=45, height=500)
    fig_decay.show()

# 4. Cohort Lifetime Value Analysis
print("4. COHORT LIFETIME VALUE ANALYSIS")
print("=" * 50)

# Calculate lifetime value metrics
lifetime_value_analysis = (
    retention_data
    .groupby('cohort_month', as_index=False)
    .agg(
        cohort_size=('cohort_size', 'first'),
        total_revenue=('total_sales', 'sum'),
        avg_order_value=('total_sales', 'mean'),
        total_orders=('unique_customers', 'sum')
    )
)

lifetime_value_analysis['lifetime_value'] =␣
 ↪lifetime_value_analysis['total_revenue'] /␣
 ↪lifetime_value_analysis['cohort_size']
```

```
lifetime_value_analysis['orders_per_customer'] =␣
 ↪lifetime_value_analysis['total_orders'] /␣
 ↪lifetime_value_analysis['cohort_size']
lifetime_value_analysis['cohort_month_str'] =␣
 ↪lifetime_value_analysis['cohort_month'].dt.strftime('%Y-%m')

print("Cohort lifetime value analysis:")
print(lifetime_value_analysis[['cohort_month_str', 'cohort_size',␣
 ↪'lifetime_value', 'orders_per_customer', 'avg_order_value']].round(2))
print()

# Visualize lifetime value trends
fig_lifetime = px.scatter(
    lifetime_value_analysis,
    x='cohort_size',
    y='lifetime_value',
    size='orders_per_customer',
    color='avg_order_value',
    title='Cohort Lifetime Value Analysis',
    labels={'cohort_size': 'Cohort Size', 'lifetime_value': 'Lifetime Value per␣
 ↪Customer ($)'},
    hover_data=['cohort_month_str', 'orders_per_customer']
)

fig_lifetime.update_layout(height=600)
fig_lifetime.show()
# 5. Final Analysis Summary (Clean Version)
print("5. FINAL ANALYSIS SUMMARY")
print("=" * 50)

# Calculate summary statistics (health_score should already be calculated in␣
 ↪previous cell)
total_cohorts = len(cohort_performance)
if 'health_grade' in cohort_performance.columns:
    high_performing_cohorts =␣
 ↪len(cohort_performance[cohort_performance['health_grade'].isin(['A', 'B'])])
    avg_health_score = cohort_performance['health_score'].mean()
else:
    high_performing_cohorts = 0
    avg_health_score = 0

# Get best and worst cohorts
best_cohort = cohort_performance.loc[cohort_performance['revenue_rank'].
 ↪idxmin()]
worst_cohort = cohort_performance.loc[cohort_performance['revenue_rank'].
 ↪idxmax()]
```

```
print("COHORT ANALYSIS COMPLETE")
print("=" * 30)
print()

print("ANALYSIS SCOPE:")
print(f"• Analyzed {total_cohorts} customer cohorts")
print(f"• Time period: {cohort_performance['cohort_month'].min().
 ↪strftime('%Y-%m')} to {cohort_performance['cohort_month'].max().
 ↪strftime('%Y-%m')}")
print(f"• Total customers: {cohort_performance['cohort_size'].sum():,}")
print(f"• Total revenue: ${cohort_performance['total_revenue'].sum():,.2f}")
print()

print("KEY METRICS:")
print(f"• Average 6-month retention rate:␣
 ↪{cohort_performance['avg_retention_6m'].mean():.1%}")
print(f"• Average revenue per customer:␣
 ↪${cohort_performance['revenue_per_customer'].mean():.2f}")
print(f"• Average cohort size: {cohort_performance['cohort_size'].mean():.0f}")
if 'health_grade' in cohort_performance.columns:
    print(f"• High-performing cohorts (A/B grade): {high_performing_cohorts}/
 ↪{total_cohorts} ({high_performing_cohorts/total_cohorts:.1%})")
    print(f"• Average cohort health score: {avg_health_score:.2f}")
print()

print("PERFORMANCE HIGHLIGHTS:")
print(f"• Best cohort: {best_cohort['cohort_month_str']} - Revenue:␣
 ↪${best_cohort['total_revenue']:,.2f}, Retention:␣
 ↪{best_cohort['avg_retention_6m']:.1%}")
print(f"• Worst cohort: {worst_cohort['cohort_month_str']} - Revenue:␣
 ↪${worst_cohort['total_revenue']:,.2f}, Retention:␣
 ↪{worst_cohort['avg_retention_6m']:.1%}")
print()

print("ANALYSIS COMPLETE - SEE EXECUTIVE SUMMARY BELOW")
print("=" * 50)
```

```
=== ADVANCED COHORT VISUALIZATIONS ===

1. COHORT PERFORMANCE DASHBOARD
==================================================

2. COHORT HEALTH SCORE HEATMAP
==================================================

3. COHORT RETENTION DECAY ANALYSIS
==================================================
```

```
Cohort retention decay analysis:
   cohort_month_str  decay_rate  initial_retention  final_retention
0           2016-10       0.002              0.003            0.048
1           2017-01       0.003              0.011            0.064
2           2017-02       0.003              0.016            0.069
3           2017-03       0.003              0.031            0.060
4           2017-04       0.002              0.026            0.063
5           2017-05       0.002              0.038            0.059
6           2017-06       0.003              0.032            0.068
7           2017-07       0.002              0.044            0.070
8           2017-08       0.002              0.041            0.063
9           2017-09       0.001              0.044            0.059
10          2017-10       0.001              0.044            0.067
11          2017-11      -0.001              0.080            0.067
12          2017-12      -0.001              0.064            0.066
13          2018-01      -0.001              0.072            0.067
14          2018-02      -0.001              0.071            0.063
15          2018-03      -0.001              0.067            0.064
16          2018-04      -0.010              0.070            0.000
17          2018-05      -0.002              0.072            0.062


4. COHORT LIFETIME VALUE ANALYSIS
==================================================
Cohort lifetime value analysis:
   cohort_month_str  cohort_size  lifetime_value  orders_per_customer
avg_order_value
0           2016-09            2          103.93                 1.00
103.93
1           2016-10          293          150.94                 0.98
2211.22
2           2016-12            1           10.90                 1.00
10.90
3           2017-01          787          151.51                 0.99
5961.83
4           2017-02         1718          139.74                 0.98
12635.09
5           2017-03         2617          135.48                 0.96
19697.47
6           2017-04         2377          139.64                 0.93
19524.42
7           2017-05         3640          126.86                 0.91
28860.71
8           2017-06         3205          114.02                 0.87
24362.19
9           2017-07         3946          105.44                 0.84
29718.98
10          2017-08         4272          106.14                 0.80
```

34879.95
11 2017-09 4227 113.67 0.76
40039.73
12 2017-10 4547 102.21 0.70
42248.26
13 2017-11 7423 90.09 0.67
66875.86
14 2017-12 5620 80.26 0.60
50119.69
15 2018-01 7187 70.59 0.54
63415.26
16 2018-02 6625 58.96 0.46
55803.44
17 2018-03 7168 52.92 0.39
63222.01
18 2018-04 6919 47.28 0.32
54520.74
19 2018-05 6833 38.21 0.26
65279.43
20 2018-06 6145 27.28 0.20
55879.32
21 2018-07 6233 18.73 0.13
58374.60
22 2018-08 6421 7.86 0.06
50483.79

5. FINAL ANALYSIS SUMMARY
==
COHORT ANALYSIS COMPLETE
==============================

ANALYSIS SCOPE:
• Analyzed 23 customer cohorts
• Time period: 2016-09 to 2018-08
• Total customers: 98,206
• Total revenue: $7,052,360.24

KEY METRICS:
• Average 6-month retention rate: 3.6%
• Average revenue per customer: $86.64
• Average cohort size: 4270
• High-performing cohorts (A/B grade): 14/23 (60.9%)
• Average cohort health score: 0.54

PERFORMANCE HIGHLIGHTS:
• Best cohort: 2017-11 - Revenue: $668,758.58, Retention: 6.7%
• Worst cohort: 2016-12 - Revenue: $10.90, Retention: 0.0%

```
ANALYSIS COMPLETE - SEE EXECUTIVE SUMMARY BELOW
================================================
```

[24]:
```
# Generate Executive Summary

from datetime import datetime

# Generate the executive summary markdown with actual calculated values
executive_summary = f"""# Executive Summary: Customer Cohort Analysis

## Analysis Overview

This comprehensive cohort analysis examines customer behavior patterns based on␣
 ↪their first purchase month, providing actionable insights for customer␣
 ↪acquisition and retention strategies. The analysis covers **{total_cohorts}␣
 ↪customer cohorts** spanning from **{cohort_performance['cohort_month'].min().
 ↪strftime('%Y-%m')}** to **{cohort_performance['cohort_month'].max().
 ↪strftime('%Y-%m')}**, representing **{cohort_performance['cohort_size'].
 ↪sum():,} total customers** and **${cohort_performance['total_revenue'].sum():
 ↪,.2f} in total revenue**.

## Key Findings

### Performance Metrics
- **Average 6-month retention rate**: {cohort_performance['avg_retention_6m'].
 ↪mean():.1%}
- **Average revenue per customer**:␣
 ↪${cohort_performance['revenue_per_customer'].mean():.2f}
- **Average cohort size**: {cohort_performance['cohort_size'].mean():.0f}␣
 ↪customers
- **High-performing cohorts (A/B grade)**: {high_performing_cohorts}/
 ↪{total_cohorts} ({high_performing_cohorts/total_cohorts:.1%})
- **Average cohort health score**: {avg_health_score:.2f}

### Cohort Performance Highlights
- **Best performing cohort**: {best_cohort['cohort_month_str']} - Revenue:␣
 ↪${best_cohort['total_revenue']:,.2f}, Retention:␣
 ↪{best_cohort['avg_retention_6m']:.1%}
- **Worst performing cohort**: {worst_cohort['cohort_month_str']} - Revenue:␣
 ↪${worst_cohort['total_revenue']:,.2f}, Retention:␣
 ↪{worst_cohort['avg_retention_6m']:.1%}

## Business Impact

### Strategic Insights
```

```
- **Cohort Performance Variation**: Significant differences in performance␣
 ↪across acquisition periods, indicating opportunities for targeted␣
 ↪interventions
- **Retention Patterns**: Clear retention decay patterns identified, with␣
 ↪critical drop-off points in months 3-6
- **Revenue Optimization**: Strong correlation between cohort size and revenue␣
 ↪per customer, suggesting scalable acquisition strategies
- **Predictive Framework**: Early cohort performance indicators can predict␣
 ↪long-term success

### Identified Opportunities
- **Targeted Interventions**: Best and worst performing cohorts identified for␣
 ↪focused improvement efforts
- **Benchmarking**: Established performance benchmarks for ongoing cohort␣
 ↪evaluation
- **Acquisition Strategy**: Successful cohort patterns can be replicated for␣
 ↪future customer acquisition
- **Retention Programs**: Critical retention periods identified for␣
 ↪intervention campaigns

## Strategic Recommendations

### Immediate Actions (0-3 months)
1. **Implement cohort-specific marketing campaigns** targeting underperforming␣
 ↪cohorts
2. **Focus on improving 6-month retention rates** through targeted retention␣
 ↪programs
3. **Replicate successful cohort acquisition strategies** from high-performing␣
 ↪periods

### Medium-term Initiatives (3-6 months)
4. **Monitor cohort health scores monthly** with automated alerting systems
5. **Develop automated cohort performance alerts** for early intervention
6. **Create cohort-based customer segmentation** for personalized experiences

### Long-term Strategy (6+ months)
7. **Implement cohort-specific retention programs** based on lifecycle stage
8. **Develop predictive models for cohort success** using machine learning
9. **Establish cohort performance as a key business metric** in executive␣
 ↪dashboards

## Implementation Framework

### Success Metrics
- **Retention Rate Improvement**: Target 15% increase in 6-month retention
```

```
- **Revenue per Customer Growth**: Target 10% increase in average revenue per␣
 ↪customer
- **Cohort Health Score**: Target 80% of cohorts achieving A/B grade performance
- **Customer Lifetime Value**: Target 20% increase in average cohort lifetime␣
 ↪value

### Monitoring and Evaluation
- **Monthly cohort performance reviews** with stakeholder reporting
- **Quarterly strategic assessment** of cohort-based initiatives
- **Annual cohort analysis refresh** with updated benchmarks and recommendations

"""

# Display the executive summary
from IPython.display import Markdown, display
display(Markdown(executive_summary))
```

2 Executive Summary: Customer Cohort Analysis

2.1 Analysis Overview

This comprehensive cohort analysis examines customer behavior patterns based on their first purchase month, providing actionable insights for customer acquisition and retention strategies. The analysis covers **23 customer cohorts** spanning from **2016-09** to **2018-08**, representing **98,206 total customers** and **$7,052,360.24 in total revenue**.

2.2 Key Findings

2.2.1 Performance Metrics

- **Average 6-month retention rate**: 3.6%
- **Average revenue per customer**: $86.64
- **Average cohort size**: 4270 customers
- **High-performing cohorts (A/B grade)**: 14/23 (60.9%)
- **Average cohort health score**: 0.54

2.2.2 Cohort Performance Highlights

- **Best performing cohort**: 2017-11 - Revenue: $668,758.58, Retention: 6.7%
- **Worst performing cohort**: 2016-12 - Revenue: $10.90, Retention: 0.0%

2.3 Business Impact

2.3.1 Strategic Insights

- **Cohort Performance Variation**: Significant differences in performance across acquisition periods, indicating opportunities for targeted interventions
- **Retention Patterns**: Clear retention decay patterns identified, with critical drop-off points in months 3-6

- **Revenue Optimization**: Strong correlation between cohort size and revenue per customer, suggesting scalable acquisition strategies
- **Predictive Framework**: Early cohort performance indicators can predict long-term success

2.3.2 Identified Opportunities

- **Targeted Interventions**: Best and worst performing cohorts identified for focused improvement efforts
- **Benchmarking**: Established performance benchmarks for ongoing cohort evaluation
- **Acquisition Strategy**: Successful cohort patterns can be replicated for future customer acquisition
- **Retention Programs**: Critical retention periods identified for intervention campaigns

2.4 Strategic Recommendations

2.4.1 Immediate Actions (0-3 months)

1. **Implement cohort-specific marketing campaigns** targeting underperforming cohorts
2. **Focus on improving 6-month retention rates** through targeted retention programs
3. **Replicate successful cohort acquisition strategies** from high-performing periods

2.4.2 Medium-term Initiatives (3-6 months)

4. **Monitor cohort health scores monthly** with automated alerting systems
5. **Develop automated cohort performance alerts** for early intervention
6. **Create cohort-based customer segmentation** for personalized experiences

2.4.3 Long-term Strategy (6+ months)

7. **Implement cohort-specific retention programs** based on lifecycle stage
8. **Develop predictive models for cohort success** using machine learning
9. **Establish cohort performance as a key business metric** in executive dashboards

2.5 Implementation Framework

2.5.1 Success Metrics

- **Retention Rate Improvement**: Target 15% increase in 6-month retention
- **Revenue per Customer Growth**: Target 10% increase in average revenue per customer
- **Cohort Health Score**: Target 80% of cohorts achieving A/B grade performance
- **Customer Lifetime Value**: Target 20% increase in average cohort lifetime value

2.5.2 Monitoring and Evaluation

- **Monthly cohort performance reviews** with stakeholder reporting
- **Quarterly strategic assessment** of cohort-based initiatives
- **Annual cohort analysis refresh** with updated benchmarks and recommendations